

Cross-Chain Content Rights Management Service

A Polkadot-Based Unified Rights Token for Subscription, Pay-Per-View, and Permanent Ownership

Ryc Brownrigg

Supervised by Dr Joshua Ellul

Centre for Distributed Ledger Technologies
Faculty of ICT
University of Malta

September, 2026

A dissertation submitted in partial fulfilment of the requirements for the degree of M.Sc. in Blockchain and Distributed Ledger Technologies (ICT).



Copyright ©2026 University of Malta

WWW.UM.EDU.MT

Fifth edition, September 22, 2026



**L-Università
ta' Malta**

Declaration by Postgraduate Students

(a) Authenticity of Dissertation

I hereby declare that I am the legitimate author of this Dissertation and that it is my original work.

No portion of this work has been submitted in support of an application for another degree or qualification of this or any other university or institution of higher education.

I hold the University of Malta harmless against any third party claims with regard to copyright violation, breach of confidentiality, defamation and any other third party right infringement.

(b) Research Code of Practice and Ethics Review Procedures

I declare that I have abided by the University's Research Ethics Review Procedures.

As a Master's student, as per Regulation 58 of the General Regulations for University Postgraduate Awards, I accept that should my dissertation be awarded a Grade A, it will be made publicly available on the University of Malta Institutional Repository.

Faculty/Institute/Centre/School	Faculty of ICT
Degree	M.Sc. in Blockchain and Distributed Ledger Technologies (ICT)
Title	Cross-Chain Content Rights Management Service
Candidate (Id.)	Ryc Brownrigg (2104001)

Signature of Student

Date

September 22, 2026

Acknowledgements

I express my deepest gratitude to my supervisor, Dr Joshua Ellul, for his insightful guidance, patience, and unwavering support throughout this dissertation.

I am also grateful to the faculty of the Centre for Distributed Ledger Technologies for creating the academic environment in which this work was produced.

To the Polkadot and Kusama communities for their documentation, tooling, and open-source contributions that supported this work. We also thank Parity Technologies for the Polkadot SDK, comprehensive documentation, and tooling that made this implementation feasible.

I am particularly grateful to Grammarly, which served as an indispensable tool for grammar, spelling, clarity, and rephrasing throughout the writing and revision process. I am also grateful to SuperWhisper, the dictation software that allowed me to compose most of this dissertation by voice rather than typing, making the writing process far more accessible and efficient given my limited typing speed.

Finally, I want to thank my mother, whose endless love, encouragement, and faith in me have carried me through all my wild and wonderful adventures. I love you so much, Mom.

Thank you all.

Abstract

The content creator economy concentrates value in centralized intermediaries. No existing system, centralized or decentralized, provides subscription access, pay-per-view consumption, and permanent ownership through a single rights primitive that is portable across heterogeneous blockchain networks.

Polkadot's architecture is well-suited to investigate this gap. Its relay chain provides shared validator security, removing the external trust assumptions inherent in bridge-based solutions; XCM natively carries rights-management operations between parachains through a standardized protocol; and FRAME's composable module system allows subscription, pay-per-view, and permanent-ownership logic to coexist within a single runtime unit. This dissertation asks: *how can a shared-security, multi-chain framework built on Polkadot XCM and FRAME primitives deliver a unified rights record natively supporting all three monetization models across heterogeneous blockchain networks, and what levels of transaction finality, cost efficiency, and creator revenue retention can it achieve?*

The Cross-Chain Content Rights Management Service (CCRMS) is a Polkadot parachain prototype designed to address this question. It has been evaluated through unit testing, performance benchmarks, a security audit, a centralized baseline comparison, and a Monte Carlo creator-revenue simulation.

The prototype achieves 26.2 transactions per second at a 96.9% inclusion rate in the happy-path window of the stress test (Section 7.1.2), approximately six-second inclusion latency, and a destination-chain inclusion delay of up to seven blocks (median 5, approximately 30–35 s). The economic simulation yields approximately 16.8% higher average creator revenue than centralized platforms, though this advantage is highly audience-dependent: creators above approximately 10,000 followers benefit decisively, whereas those below approximately 3,600 are better served by centralized discovery platforms. This result characterizes the CCRMS design; the benchmarked prototype bounds the number of rights holders per content item, which must be lifted before the advantage can be realized for larger audiences.

Three findings advance the state of knowledge. First, a unified rights record supporting all three access models can be built from standard FRAME primitives without novel cryptographic mechanisms, closing literature gap G7. Second, the revenue advantage of decentralized rights management is not universal, crossing a follower threshold at approximately 3,600. Third, conservative architectural choices provided resilience against an ecosystem-level disruption that occurred during the implementation period.

Keywords: cross-chain content rights, Polkadot XCM, unified rights record, creator revenue retention, design science research

Contents

List of Abbreviations	xix
1 Introduction	1
1.1 The Problem	1
1.2 Background Context	2
1.3 Research Aim and Approach	3
1.4 Contributions	4
1.5 Scope, Claims, and Limitations	4
1.5.1 Claims	5
1.5.2 Non-Claims	5
1.5.3 Operational Limitations	6
2 Background and Related Work	7
2.1 Blockchain-Based Content Management	7
2.1.1 Market Context	7
2.1.2 Foundational Work on Blockchain-Based DRM	7
2.1.3 NFTs and Intellectual Property Rights	8
2.2 Cross-Chain Interoperability	8
2.2.1 Polkadot Architecture	9
2.2.2 Cross-Chain Technology Surveys	9
2.2.3 Bridges and Snowbridge	9
2.2.4 Polkadot 2.0 and JAM	10
2.2.5 Cross-Chain Copyright Management as a Research Target	10
2.3 Decentralized Monetization Models	11
2.3.1 Creator Economy Context	11

2.3.2	DeFi and Web3 Foundations	11
2.3.3	Subscription, Pay-Per-View, and Purchase Models	11
2.3.4	Pay-Per-View and the Micropayment Problem	11
2.3.5	Smart Contracts as the Implementation Layer	12
2.3.6	Royalty Automation	12
2.4	Challenges and Gaps	12
2.5	Summary	16
3	Research Questions and Objectives	17
3.1	Primary Research Question	17
3.2	Sub-Questions	18
3.3	Research Objectives	19
3.4	Mapping Questions to Objectives	20
3.5	Summary	21
4	Methodology and System Design	23
4.1	Research Methodology	23
4.1.1	Design Science Research Framing	23
4.1.2	Iterative Prototyping	24
4.1.3	Data Collection Methods	24
4.1.4	Evaluation Philosophy: Measurement-Focused	24
4.1.5	Methodology Pivots	25
4.2	System Design	25
4.2.1	The Unified Rights Record Concept	25
4.2.2	The Three Monetization Models	25
4.2.3	NFT-Based Rights Tokenization	26
4.2.4	Cross-Chain Communication Model	28
4.2.5	Pallet-First Architecture	29
4.2.6	Layered Architecture	29
4.3	Summary	30
5	Implementation	31
5.1	Pallet Implementation and Smart Contract Layer	31
5.2	Cross-Chain Integration	32
5.2.1	XCM Message Flows	32
5.2.2	Snowbridge Integration with Ethereum	33
5.3	Challenges and Solutions	34
5.4	Summary	36

6	Evaluation and Testing	37
6.1	Evaluation Approach	37
6.2	Testing Layers	38
6.3	Performance Testing Framework	38
6.4	Test Scenarios	39
6.5	Metrics and Key Performance Indicators	41
6.6	Testing Infrastructure	43
6.7	Validity and Limitations	43
6.8	Summary	44
7	Results	45
7.1	Technical Results	45
7.1.1	Throughput	45
7.1.2	Sustained Throughput (Stress Test)	46
7.1.3	Latency	46
7.1.4	Block Weight Utilization	47
7.1.5	Storage Growth	47
7.1.6	Reliability	48
7.1.7	Snowbridge End-to-End Bridge	49
7.1.8	Comparative Analysis vs. Centralized Baseline	49
7.2	Economic Results	50
7.2.1	Monte Carlo Creator Revenue Simulation	50
7.2.2	Audience Size Dependency	51
7.2.3	Comparative Fee Analysis	53
7.2.4	Sensitivity Analysis	53
7.2.5	Prototype-Bound versus Simulated Cohorts	54
7.3	Security and User-Centric Results	54
7.3.1	Security Audit Findings	54
7.3.2	Test Coverage	55
7.3.3	Decentralization Assessment (HHI)	56
7.3.4	Static Analysis and Dependency Audit	57
7.4	Summary	57
8	Discussion	59
8.1	Answering the Research Questions	59
8.1.1	SQ1: Cross-Chain Rights Operations and Finality	59
8.1.2	SQ2: Latency versus Decentralization	60

8.1.3	SQ3: Unified Rights Record	60
8.1.4	SQ4: Security and Authorization	60
8.1.5	SQ5: Economic Viability	60
8.2	Interpretation of Key Results	61
8.2.1	The 26.2 TPS Figure in Context	61
8.2.2	The Economic Crossover at ~3,600 Followers	61
8.3	Architectural Decisions Revisited	62
8.4	The Work in Context	62
8.5	Limitations	62
8.6	Implications	63
8.7	Critical Self-Reflection	64
8.8	Summary	64
9	Conclusion and Future Work	65
9.1	Summary of Contributions	65
9.2	Principal Findings	66
9.3	Future Work	67
9.4	Closing Reflection	69
Appendix A	Implementation Challenge Narratives	71
A.1	The Pivot from RMRK 2.0 to pallet-nfts	71
A.2	Contract-First to Pallet-First Pivot	72
A.3	The Snowbridge Fork-Version Configuration Issue	72
A.4	The Lodestar Preset Version Pinning	73
A.5	The XCM Sovereign Account Endianness Bug	73
A.6	The XCM Fee Scaling for Proof Size	74
A.7	Finding B: Authorization Gap	74
A.8	Auto-Renewal Child Cap	74
A.9	ink! Discontinuation	75
Appendix B	Research Objective Details	77
B.1	RO1: Design and Prototype XCM Extensions	77
B.2	RO2: Implement a Unified Rights Token	77
B.3	RO3: Merge Subscription, PPV, and Purchase Logic	77
B.4	RO4: Benchmark Scalability and Interoperability	78
B.5	RO5: Zero-Knowledge Proof Hooks (Not Achieved)	78
B.6	RO6: Quantify Creator Savings	78

Appendix C Code Listings	79
C.1 Solidity Interface: ContentRightsPrecompile	79
C.2 Canonical XCM Message	80
C.3 Repository Directory Layout	80
C.4 Dispatchable Extrinsic Reference	81
C.5 Pallet Storage Layout Reference	82
Appendix D Security and Coverage Findings	85
D.1 Finding B: Authorization Gap	85
D.2 Security Audit Finding E: View Pack Overwrite	86
D.3 Accepted Design Findings (A, C, D)	86
D.4 Finding I: Zero-View Denial of Service	86
D.5 Error Variant Coverage	87
D.6 Static Analysis Advisory Breakdown	87
Appendix E Future Work Details	89
E.1 Production Deployment	89
E.2 Migration to Polkadot 2.0 Features	89
E.3 Migration Paths for the Smart Contract Layer	90
E.4 The 50-Child Cap	90
E.5 User Study and Consumer Application	90
E.6 The Bridged-Payment Final Hop	91
E.7 Resale Royalties	91
E.8 Capability-Based Delegation	91
E.9 Monte Carlo Simulation Extensions	91
E.10 Formal Verification	91
E.11 JAM Transition	92
Appendix F Work Plan and Timeline	93
Appendix G Monte Carlo Simulation Parameters and Sensitivity	95
References	99

List of Tables

2.1	Literature Gaps and CCRMS Contributions	12
3.1	Research Sub-Questions	18
3.2	Research Objectives Summary	19
3.3	Mapping of Sub-Questions to Objectives, Methods, and Chapters	20
5.1	Implementation Challenges Summary	34
6.1	Mapping of Evaluation Activities to Research Sub-Questions	37
6.2	Benchmark Scripts	38
6.3	Test Scenarios	39
6.4	KPI Definitions	42
6.5	Test Environment Specifications	43
7.1	Single-Chain Inclusion Capacity at Batch Size 100	46
7.2	XCM Destination-Chain Inclusion Delay by Operation ($n = 3$ per operation) .	47
7.3	Block Weight Utilization at Increasing Batch Sizes	47
7.4	Per-Item Storage Costs (SCALE-encoded)	48
7.5	Reliability Test Results	48
7.6	Snowbridge End-to-End Message Path (ETH Transfers to AssetHub; CCRMS Not Exercised)	49
7.7	Comparative Analysis: CCRMS Parachain vs. Centralized (Express.js + SQLite)	50
7.8	Monte Carlo Creator Revenue Simulation (Mean Annual Revenue Per Creator)	51
7.9	CCRMS Win Rate Against Centralized Platform by Audience Tier	52
7.10	Per-Scenario Break-Even Rates	52
7.11	95% Confidence Intervals for Total Simulated Revenue (1,000 Creators)	53

7.12	Published fee ranges by platform category. Centralized literature values span 30–60%; the simulation uses 30–45% (Appendix G, Table G.2).	53
7.13	Security Audit Findings	54
7.14	Herfindahl-Hirschman Index (HHI) Comparison	56
7.15	KPI Results Summary	57
9.1	Future Work Summary	67
C.1	Dispatchable Extrinsic in pallet-content-rights	81
C.2	Storage Layout of pallet-content-rights	82
D.1	Error Variant Coverage	87
D.2	Static Analysis Advisory Breakdown	87
G.1	Monte Carlo Stochastic Parameters (per creator, per iteration)	95
G.2	Monte Carlo Platform Parameters (drawn uniformly once per iteration)	96
G.3	Sensitivity of CCRMS Advantage to Individual Parameter Changes	96
G.4	Sensitivity of CCRMS Advantage to Fee Assumption	97
G.5	Sensitivity of CCRMS Advantage to Centralized Discovery Assumption	97
G.6	Fine-Grained CCRMS Win Rate Near the Crossover Audience Size	98

List of Figures

4.1	The three monetization models natively supported by the unified rights record (addresses SQ3). A single content registration may offer all three models concurrently; <code>check_access</code> resolves in priority order: Ownership > Subscription > PPV.	26
4.2	Parent-child NFT nesting model. Each content registration creates one NFT collection containing a parent Content NFT (the canonical on-chain identity of the content) and zero or more child rights NFTs, each owned by an individual user and tagged with their access mode.	27
4.3	XCM cross-chain rights operation (addresses SQ1 and SQ4). The three-instruction sequence dispatches an <code>xcm_*</code> extrinsic on the CCRMS parachain, decoupling the paying account (Para B sovereign account) from the rights beneficiary (the end user). The same pattern applies to all five <code>xcm_*</code> extrinsics.	28
4.4	CCRMS six-layer architecture (addresses SQ1 and SQ2). Each layer is independently replaceable: Layer 5 (smart contracts) may be removed, Layer 6 (XCM) may upgrade from v3 to v5, and Layer 1 (relay chain) may migrate to JAM, without modifying the business logic in Layer 3.	30
5.1	CCRMS implementation-level component diagram. Solid arrows indicate active extrinsic dispatch; dashed arrows indicate read-only storage access. <code>pallet-content-rights</code> is the sole authoritative state layer; the <code>ink!</code> contract and EVM precompile are stateless wrappers that forward calls or read storage, respectively. XCM delivers cross-chain calls via HRMP. <code>pallet-rights-verifier</code> enables remote parachains to validate rights state via Merkle storage proofs, without oracles or relayers.	32

- 6.1 Four-layer testing architecture (Layer 1 at base: fastest and most granular, run on every commit; Layer 4 at apex: broadest scope, run infrequently). Each layer tests what lower layers cannot: unit tests isolate individual extrinsic logic; the XCM simulator isolates message-construction failures; Zombienet provides definitive wall-clock performance evidence; Snowbridge validates the complete Ethereum bridge pathway. 41
- 7.1 Monte Carlo simulation structure: 1,000 creators $\times$ 10,000 iterations = 10 million revenue paths per platform. Platform fee drawn uniformly each iteration; full parameters in Appendix G. 51

List of Abbreviations

ABI	Application Binary Interface
AI	Artificial Intelligence
API	Application Programming Interface
ASCII	American Standard Code for Information Interchange
BLS	Boneh–Lynn–Shacham (signature scheme)
CAGR	Compound Annual Growth Rate
CCRMS	Cross-Chain Content Rights Management Service
CLI	Command-Line Interface
CPU	Central Processing Unit
DEX	Decentralized Exchange
DOT	Polkadot native token
DRM	Digital Rights Management
DSR	Design Science Research
ESMA	European Securities and Markets Authority
ETH	Ether (Ethereum’s native token)
EU	European Union
EVM	Ethereum Virtual Machine
FRAME	Framework for Runtime Aggregation of Modularized Entities
GB	Gigabyte
H160	160-bit hash (Ethereum-style address)
HHI	Herfindahl-Hirschman Index
HRMP	Horizontal Relay-routed Message Passing
HTTP	Hypertext Transfer Protocol
ID	Identifier

IPFS	InterPlanetary File System
JAM	Join-Accumulate Machine
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
MEV	Maximal Extractable Value
MiCA	Markets in Crypto-Assets (EU Regulation)
MTTR	Mean Time To Recovery
NFT	Non-Fungible Token
OTT	Over-The-Top (media service)
PIL	Programmable IP License (Story Protocol)
PolkaVM	Polkadot Virtual Machine
PPV	Pay-Per-View
RISC	Reduced Instruction Set Computer
RMRK	Remark (NFT protocol; derived from the <code>system.remark</code> extrinsic)
RPC	Remote Procedure Call
SCALE	Simple Concatenated Aggregate Little-Endian (Substrate codec)
SDK	Software Development Kit
SIGTERM	Signal Terminate (POSIX signal)
SQL	Structured Query Language
TLS	Transport Layer Security
TPS	Transactions Per Second
TVL	Total Value Locked
UI	User Interface
USD	United States Dollar
WASM	WebAssembly
XCM	Cross-Consensus Messaging
ZK	Zero-Knowledge (proof / cryptography)

Introduction

1.1 | The Problem

The prevailing content economy is unfavorable to creators. An independent musician earning USD 100,000 in streaming revenue typically retains between USD 55,000 and USD 70,000 after platform fees and distributor commissions — a figure that is substantially lower for label-signed artists (Music Business Research, 2024). Centralized intermediaries retain between 30 and 60 percent of gross creator revenue, depending on the platform and contract structure (the per-platform breakdown is documented in Table 7.12). Fees are non-negotiable, audiences are non-portable, and content rights remain bound by the platform's terms of service.

Consumers face similar fragmentation. A film purchased on iTunes cannot be streamed on Amazon Prime; an Audible audiobook cannot be transferred to Google Play; a subscription started on one platform cannot be moved to another. Consumers do not own their purchases; they hold a platform-bound license that can be revoked, modified, or ignored. This fragmentation that constrains creators also restricts the consumers who support them.

Fragmentation extends to monetization models. Subscriptions typically live on Patreon; one-time sales on Gumroad; pay-per-view is scarcely supported because the overhead of conventional payment systems makes micro-transactions uneconomic. Each platform supports a single monetization model, has its own audience, charges its own fees, and produces its own accounting reports. A creator who wants to offer all three models is forced to run three or four platforms in parallel and reconcile them by hand.

Existing blockchain alternatives address only parts of the problem, not the whole. Non-fungible tokens (NFTs) provide transferable ownership but not recurring access or per-view consumption. Subscription-style decentralized platforms such as Audius

offer recurring revenue but are confined to a single chain and audience. Cross-chain bridges move assets between ecosystems but carry no rights metadata during those transfers. Academic work on blockchain-based digital rights management has produced tamper-evident provenance schemes (Garba et al., 2021; Patil et al., 2022), expert-derived design principles for royalty distribution in the music industry (Ciriello et al., 2023), and on-chain licensing frameworks (Mendoza-Tello et al., 2024) — each addressing a single dimension of the problem on a single chain. No existing system, centralized or decentralized, combines subscription access, pay-per-view consumption, and permanent ownership in a single rights primitive that is portable across blockchains.

The technical primitives needed to address this gap already exist as standalone capabilities: cross-chain messaging, programmable smart contracts, on-chain royalty distribution, and NFT-based rights tokenization. Hanneke et al. (2024) characterize this class of systems as an emerging *Internet of Value*, in which blockchain tokenization enables asset ownership and revenue flows that bypass centralized intermediaries entirely. However, no prior academic or industry work has integrated these primitives into a system that unifies all three monetization models in a single, cross-chain-portable rights record — the specific gap that Chapter 2 maps across eight dimensions of prior literature (Table 2.1). The research questions that follow from this gap are formalized in Chapter 3.

1.2 | Background Context

This dissertation sits at the intersection of three active research and engineering areas: blockchain-based digital rights management, cross-chain interoperability, and decentralized content monetization. Chapter 2 surveys each area in depth; this section summarizes what has been proposed in those areas and explains why no existing proposal meets the combined requirements.

Story Protocol (now Data Foundation), launched in February 2025, is the closest industry precedent: an EVM-compatible layer-1 blockchain that records IP ownership and licensing terms on-chain and supports programmable royalty flows (Story Foundation, 2025). Academic proposals in this domain include tamper-evident provenance schemes, royalty-distribution principles, and on-chain licensing frameworks, each targeting a single aspect of rights management on a single chain (surveyed in detail in Chapter 2). The consistent limitation is that all such systems operate on a single chain and address either content protection or royalty distribution, but not the full combination of subscription, pay-per-view, and permanent-ownership access models.

Cross-chain interoperability has emerged as an active research area. A 2023 survey

(Ren et al., 2023) classifies interoperability approaches into five categories, and two additional 2025 surveys (Deng et al., 2025; Li et al., 2025) collectively cover more than 150 sources. These surveys examine mechanisms for moving assets between ecosystems but do not address the portability of rights metadata during those movements. That is the specific contribution of this dissertation’s cross-chain design.

These architectural properties are examined in depth in Chapter 2, and their technical implications for the CCRMS design are discussed in Chapter 4.

1.3 | Research Aim and Approach

The gap identified in Section 1.1 and mapped in Chapter 2 (Table 2.1, gap G7) remains unaddressed by prior academic or industry work: no existing system unifies subscription access, pay-per-view consumption, and permanent ownership into a single, portable rights primitive across heterogeneous blockchain networks. The primary research question this dissertation addresses is:

How can a shared-security, multi-chain framework built on Polkadot’s Cross-Consensus Messaging (XCM) and FRAME pallet primitives deliver a unified rights record that natively supports recurring subscriptions, pay-per-view microtransactions, and permanent ownership transfers across heterogeneous blockchain networks, and what levels of transaction finality, cost efficiency, and creator revenue retention can it achieve?

The question has two mutually constraining components, design feasibility and measurement, which are fully decomposed in Chapter 3.

To address this question, we design, implement, and evaluate a working prototype, the Cross-Chain Content Rights Management Service (CCRMS), built as a Polkadot parachain. The prototype supports all three monetization models through a single unified rights record, enabling automatic royalty distribution and cross-chain portability via XCM. The rationale for choosing Polkadot as the implementation platform is presented in Section 1.2 and examined in technical detail in Chapter 4.

Our research approach is grounded in design science research (Hevner et al., 2004) and applied to a multi-component blockchain prototype. We organize the methodology, detailed in Chapter 4, into iterative build-test-measure-refine cycles within a six-phase framework. Our evaluation strategy, as delineated in Chapter 6, combines quantitative measures (including technical benchmarks, security audits, comparative analyses, and Monte Carlo simulations of creator revenue) with qualitative insights from a literature

synthesis. We frame the evaluation as measurement-focused rather than target-driven: our objective is to document and analyze the levels of performance, security, and economic viability achieved by the system, rather than to verify conformity with a predefined set of criteria.

1.4 | Contributions

We make four contributions, described in detail in Section 9.1:

1. **A feasibility result for the unified rights primitive.** This dissertation demonstrates that a single FRAME pallet with a unified priority-resolution access check across three storage maps is sufficient to encode subscription, pay-per-view, and permanent ownership as native access modes for a single token type — without novel cryptographic or consensus-layer mechanisms. This closes the gap G7 identified in Chapter 2.
2. **Quantified decentralization–latency and revenue trade-offs.** The latency cost of decentralization is measured directly: CCRMS incurs a $67,000\times$ overhead relative to a centralized baseline. The creator-revenue advantage is real but conditional: CCRMS produces higher mean revenue only for creators above approximately 3,600 followers (the 50% crossover point), with the transition beginning near 2,000 followers where the win rate reaches 6.6%.
3. **A cross-chain rights artifact demonstrating XCM sufficiency.** The CCRMS parachain — comprising two FRAME pallets, a smart contract layer, an EVM precompile, and benchmarking and simulation scripts — demonstrates that XCM v3 is sufficient for authenticated cross-chain rights operations without custom protocol extensions. The artifact is licensed under the MIT license.
4. **A DSR worked example for multi-component blockchain prototypes.** Applying design science research to a system spanning FRAME pallets, smart contracts, XCM, and Monte Carlo simulation is rarely illustrated in the existing DSR literature; this dissertation serves as a reference template for future researchers.

1.5 | Scope, Claims, and Limitations

This section records what the dissertation demonstrates, what it deliberately does not claim, and the operational constraints that bound how the results should be read.

1.5.1 | Claims

The following results are demonstrated only within the operational scope of Section 1.5.3:

1. **Unified access encoding.** A FRAME pallet can encode subscription, pay-per-view, and permanent ownership as three native access modes within a single content registration, resolved by a single `check_access` call. Demonstrated by the implementation and its unit test suite.
2. **XCM v3 sufficiency for rights issuance.** The sequence `WithdrawAsset` → `BuyExecution` → `Transact` is sufficient for issuance-style cross-parachain rights operations with payer–beneficiary decoupling in a local HRMP topology. No custom XCM extensions are required. Cross-chain transfer of existing ownership is constrained by the Finding B authorization fix (Sections 7.3.1 and 8.1.4) and is not claimed as a completed portable-secondary-market path.
3. **Audience-conditional revenue advantage.** Under the assumptions documented in Appendix G, creator revenue after fees from CCRMS exceeds centralized take rates once a creator’s audience exceeds approximately 3,600 followers. This is a design-level simulation result, not a measurement of the deployed prototype.
4. In the Zombienet topology described in Section 1.5.3, the prototype met ten of eleven KPI targets in Table 7.15; KPI 9 is met only as an author-audit interpretation (Section 7.4). Those figures characterize the test environment.

1.5.2 | Non-Claims

The dissertation does not claim, assert, or imply:

- **Production throughput, latency, or bridge behavior.** All measurements are from a local Zombienet configuration and are not production forecasts.
- **End-to-end Ethereum → CCRMS rights purchase.** Snowbridge was exercised only as `ETH` → `AssetHub`; the path to a CCRMS rights operation has not been tested end-to-end.
- **Cross-chain atomicity or auto-renewal.** XCM `Transact` does not guarantee atomicity across chains, and cross-chain recurring renewals depend on XCM v5 `Schedule`, which is not used here.

- **The prototype can host the audience tiers that win in the simulation.** The `BoundedVec<_, ConstU32<50>>` lifetime cap limits each content item to fifty subscribers and owners in total, placing the high-follower audience tiers beyond the current instantiation.
- **Decentralization is universally better for creators.** The simulation finds a CCRMS advantage only above approximately 3,600 followers; below roughly 2,000 followers, algorithmic discovery on centralized platforms yields higher expected revenue.

1.5.3 | Operational Limitations

- **The prototype is a back-end service, not a consumer application.** We did not develop a web or mobile interface for CCRMS. Our assessment focuses on pallet- and runtime-level behavior; we do not directly evaluate user experience. We recognize this as a limitation in Chapters 6 and 8, and as an area for future research in Chapter 9.
- **We measure performance on a local development network.** Our CCRMS prototype runs in a local Zombienet network (a tool for simulating multi-chain Polkadot environments during development) with two relay-chain validators and single-collator parachains. In production, parachains are hosted on the Polkadot or Kusama relay chains, which have hundreds of validators, real-world network latency, and competition from many other parachains. The throughput, latency, and reliability metrics in Chapter 7 should be treated as **upper bounds for the test environment**, not as forecasts of real-world deployment performance.
- **Our Snowbridge integration uses a local Ethereum stack.** We successfully demonstrated the bridge with two transactions, each transferring one ETH from a local Ethereum chain to a local AssetHub parachain. However, the entire configuration remains local; we have not used any public testnet or mainnet. This integration demonstrates architectural feasibility rather than production validation.
- **We have not commissioned any third-party audit, security review, or independent benchmarking.** A single author conducted the implementation, test design, measurements, and interpretation for this dissertation. Deployment to production would require third-party validation before any substantive value is processed.

Background and Related Work

This chapter surveys four themes: blockchain-based content management (Section 2.1), cross-chain interoperability (Section 2.2), decentralized monetization models (Section 2.3), and the challenges and gaps that motivate the research (Section 2.4).

2.1 | Blockchain-Based Content Management

2.1.1 | Market Context

Three independent market research firms (Grand View Research, 2025; MarketsandMarkets, 2025; Mordor Intelligence, 2026) converge on estimates that the global DRM market is valued at approximately USD 6.7–6.9 billion in 2025–2026, with projections reaching USD 11–14 billion by 2030–2033, representing a Compound Annual Growth Rate (CAGR) of 10–11%. This growth is primarily driven by streaming services, AI-enabled piracy, and regulatory pressures. The market is dominated by **content protection** solutions (such as Widevine, FairPlay, PlayReady), rather than **content monetization**.

2.1.2 | Foundational Work on Blockchain-Based DRM

Blockchain-based Digital Rights Management (DRM) is an active research domain. Garba et al. (2021) incorporate digital watermarking into blockchain technology as a tamper-evident layer; Patil et al. (2022) build on this by introducing the DRMChain hybrid encryption architecture; and Yi et al. (2023) further extend the protection model with a redactable blockchain scheme that combines DRM with perceptual hashing, enabling selective content modification without invalidating ownership proofs. Both earlier studies prioritize protection mechanisms over the economics of content creators.

In the music industry, Ciriello et al. (2023) derive a set of design principles for blockchain-based DRM systems through expert interviews and prototype testing, identifying transparency of rights ownership, automated royalty distribution, and programmable licensing as the three foundational requirements. Cherdakov et al. (2023) and Mendoza-Tello et al. (2024) propose Web3 implementations that embed these principles in music-streaming smart contracts, demonstrating the feasibility of on-chain licensing but stopping short of cross-chain portability.

The most comprehensive recent work is Madapati and Pradhan (2025), whose Ethereum + IPFS framework achieves 409 TPS for registration; this comparison is misleading because their pipeline hashes off-chain and stores only results on-chain. Wang et al. (2025) present a BLS threshold signature scheme achieving 40 ms aggregation. Two Polkadot-specific studies (Xie and Liu, 2022; Xie and Tang, 2024) propose conceptual cross-chain frameworks without working prototypes.

This literature establishes **blockchain-based DRM as a legitimate research domain**; its consistent limitation is that it emphasizes protection over monetization, a priority CCRMS reverses.

2.1.3 | NFTs and Intellectual Property Rights

Work on NFTs and intellectual property encompasses legal analysis (Davik, 2025; Eurojust, 2024; Murray, 2022; USPTO, 2024) and standardization proposals (Dreyer et al., 2021; Gatto et al., 2021; Reggianini, 2025). The most empirically significant study is Darshan et al. (2025): **80% of mainstream NFTs fail to transfer copyright, only 12–30% of listings include licensing information, and 50–60% of secondary sales bypass creator royalties**. Lahiri et al. (2026) confirm these findings via a meta-analysis of 125 NFT studies, identifying legal clarity around ownership transfers as the field’s primary unsolved problem. Cabot-Nadal et al. (2022) propose an ERC-721 extension enabling recipients to reject unwanted transfers, relevant to our permanent-ownership mode.

The most significant industry development is the February 2025 launch of **Story Protocol**, the first dedicated intellectual-property Layer-1 blockchain (Story Foundation, 2025). Built on the Cosmos SDK with EVM compatibility, it enables programmable IP registration, automated licensing, and on-chain royalty distribution. It is the most direct industry comparator to CCRMS: a single-chain L1 versus a cross-chain parachain.

2.2 | Cross-Chain Interoperability

2.2.1 | Polkadot Architecture

Polkadot (Wood, 2016) is a heterogeneous multi-chain framework in which parachains share security through a central relay chain and communicate via XCM. The core primitives are parachains, the relay chain, validators, collators, and XCM. Habermeier et al. (2021) and Wood (2022) provide the primary technical documentation for the XCM message format and multi-hop features that underpin our cross-chain implementation. The XCM protocol itself has received dedicated academic attention: Morháč et al. (2023a) build ParaSpell, a developer SDK that abstracts XCM message construction across parachains, and Morháč et al. (2023b) demonstrate XCMP interoperability at the protocol level. Together with Morháč et al. (2025), which proposes the Unispell universal adapter, they form the most coherent body of peer-reviewed academic work on XCM in practice.

Bridge-based systems such as Wormhole and LayerZero rely on external validator committees that could be compromised independently, enabling forged ownership proofs or hijacked royalty distributions. Polkadot’s shared-security model eliminates this risk: all parachains use the same relay-chain validator set. Cross-chain rights operations use HRMP, which routes messages through the relay chain and inherits its security and finality.

2.2.2 | Cross-Chain Technology Surveys

The cross-chain literature matured rapidly from 2022 to 2025. Ren et al. (2023) classify interoperability solutions into five categories: sidechains, notary schemes, hashed time lock contracts, relays, and blockchain-agnostic protocols. Mao et al. (2022) provide an earlier three-category taxonomy; Bigiotti et al. (2025) extend both in ACM Computing Surveys. Two more recent surveys expand the field: **Deng et al. (2025)** analyze more than 150 sources; **Li et al. (2025)** distinguish **asset interoperability** (token transfers) from **data interoperability** (cross-chain state verification). The latter distinction is directly relevant: CCRMS addresses both through XCM payments and pallet-rights-verifier Merkle proofs. Kumar et al. (2025) and Cao et al. (2025) examine cross-chain efficiency and trustless interoperability, both aligned with CCRMS’s design intent.

2.2.3 | Bridges and Snowbridge

Bridge architecture research includes Yin et al. (2022) on multi-party computation, Chen et al. (2023) on reputation-based trust, and Maric et al. (2025) on Isabelle/HOL formal verification of fail-safe bridges.

The most operationally significant development in blockchain infrastructure is **Snowbridge**, a trust-minimized bridge that enables interoperability between Polkadot and Ethereum. Snowbridge V2 was officially launched on the Polkadot mainnet on November 7, 2025, and includes support for arbitrary Ethereum contract calls from Polkadot, a 50% fee reduction, and unordered message execution (Snowfork, 2025a). The associated Polkasassembly proposal (Snowfork, 2025b) documents more than a year of operational stability with no on-chain downtime. We position our Snowbridge integration as forward-compatible with V2, even though we developed the initial prototype against V1.

2.2.4 | Polkadot 2.0 and JAM

Two major ecosystem developments shaped our implementation timeline. **Polkadot 2.0** was comprehensively launched by October 2025, introducing Async Backing (eliminating relay-chain wait), Agile Coretime (on-demand block production), Elastic Scaling (multiple cores per parachain), and XCM v5 (multi-hop transactions) (Parity Technologies, 2025; Polkadot Network, 2025). **JAM** (Join-Accumulate Machine, Wood (2024)) is expected to succeed the relay-chain model within the Polkadot ecosystem, with its testnet launched in January 2026 and mainnet scheduled for mid-to-late 2026. These developments confirm that the Polkadot architecture is **maturing rather than stagnating**.

2.2.5 | Cross-Chain Copyright Management as a Research Target

Numerous studies examine cross-chain copyright management as a research issue (Buzu, 2021; Mareckova, 2024; Munson et al., 2022; Orero et al., 2023; Xie and Liu, 2022; Xie and Tang, 2024); however, none have developed a functional end-to-end prototype. This gap, together with the architectural analysis above, motivates the selection of XCM and FRAME as the implementation substrate and justifies choosing Polkadot as the platform over single-chain or bridge-based alternatives.

XCM is preferred over Wormhole and LayerZero because it is native to the relay chain and introduces no additional trust assumptions. Its structured instruction format also encodes rights operations as typed, on-chain-interpretable messages rather than opaque byte payloads, preserving rights metadata across chain boundaries (gap G3). FRAME is the implementation framework of choice over EVM and CosmWasm alternatives because its composable module system allows all three access modes to share a single pallet and access-check function, its weight benchmarking provides deterministic fee calculations, and its typed storage API exposes rights state as queryable on-chain objects via the RPC layer.

2.3 | Decentralized Monetization Models

2.3.1 | Creator Economy Context

CommuniPass (2026) forecasts the creator economy will exceed USD 280 billion by the end of 2026. The blockchain-specific segment is growing rapidly, from USD 499.52 million (2024) to USD 2.74 billion by 2030 (360iResearch, 2025), driven by creator dissatisfaction with centralized platforms and maturing blockchain infrastructure.

2.3.2 | DeFi and Web3 Foundations

The conceptual foundation for decentralized monetization lies in DeFi. Gogel et al. (2021) present the canonical Wharton overview of the DeFi ecosystem, covering AMMs, lending, derivatives, and stablecoins; Atzori (2017) analyzes decentralized governance through a political economy lens; and Busch (2022) provides a technical overview of Web3 from the United States Congressional Research Service.

2.3.3 | Subscription, Pay-Per-View, and Purchase Models

Earlier work on blockchain subscription and pay-per-view models includes Banerjee et al. (2020); Castweet Team (2019); Khan (2024); Munson (2019); Steem Team (2025). Most of these works examine blockchain incentives without addressing the micropayment problem. Two more recent works are particularly relevant: **Osemwegie (2025)** surveys decentralized media monetization (identifying the problems CCRMS targets but not proposing a working system), and **Shah (2025)** proposes a smart-contract framework for royalty distribution that is the closest to our royalty propagation design in the existing literature. CCRMS extends Shah’s approach using a FRAME pallet with stronger atomicity guarantees. Wamugo (2024) provides empirical data on NFT adoption by media organizations in Kenya, demonstrating that blockchain monetization is under active consideration in emerging markets.

2.3.4 | Pay-Per-View and the Micropayment Problem

Pay-per-view models have historically struggled on blockchain platforms because gas fees have often exceeded content costs. Goyal et al. (2019) introduce Gringotts, a secure micropayment system, but do not address the core fee issue. Chegenizadeh et al. (2024) directly address the micropayment verification layer, proposing a multi-agent, privacy-preserving approach for resource-constrained offline devices that reduces per-transaction

overhead through smart-contract batching; their work supports our strategic decision to model PPV as a counter rather than a sequence of individual on-chain transactions.

2.3.5 | Smart Contracts as the Implementation Layer

Surveys of the smart contract literature (Ante, 2020; Lin et al., 2022; Quan et al., 2024; Singh et al., 2024) establish the field as mature; Parity’s ink! blog posts (Parity Technologies, 2022a,b) were the primary technical references during implementation. The most consequential development is the **discontinuation of ink! in January 2026** (ink! Alliance, 2026), discussed as gap G8 and in Appendix A.9.

2.3.6 | Royalty Automation

Few peer-reviewed studies address on-chain royalty automation; Shah (2025) and Reggianini (2025) are the primary examples (discussed in Section 2.3.3).

2.4 | Challenges and Gaps

Table 2.1 maps eight literature gaps to CCRMS contributions.

Table 2.1. Literature Gaps and CCRMS Contributions

ID	Gap	Supporting literature	CCRMS contribution
G1	Existing systems are protection-focused, not monetization-focused	Garba et al. (2021); Patil et al. (2022); Ciriello et al. (2023); Madapati and Pradhan (2025); commercial DRM (Widevine, FairPlay, PlayReady)	Unified rights token with monetization as the primary design goal; protection becomes a secondary consequence of correct rights management.

Continued on next page

Table 2.1 continued from previous page

ID	Gap	Supporting literature	CCRMS contribution
G2	No native cross-chain recurring payments	Existing cross-chain protocols (XCM, Wormhole, LayerZero, Snowbridge) address one-time transfers; Xie and Liu (2022); Xie and Tang (2024) propose theoretical frameworks without recurring payments	On-chain on_initialize block hook scheduler for automatic renewal. XCM v5 Schedule is not yet production-ready; we defer the implementation of true cross-chain scheduling to future work.
G3	Cross-chain protocols strip rights metadata on transfer	Reggianini (2025) addresses part of the standardization problem within European NFT licensing	query_rights_metadata extrinsic emitting a self-describing RightsMetadata event; metadata canonical on CCRMS, authenticated by pallet-rights-verifier Merkle proofs.
G4	Pay-per-view is economically unviable at micro-transaction scale on traditional blockchains	Goyal et al. (2019); Chegenizadeh et al. (2024); Khan (2024); x402 (Coinbase and x402 Foundation, 2025)	PPV reconceptualized as a counter: one payment per pack, cheap storage updates per view, with additive purchases (Appendix C.4).

Continued on next page

Table 2.1 continued from previous page

ID	Gap	Supporting literature	CCRMS contribution
G5	The decentralization–latency trade-off is poorly characterized quantitatively	Most works endorse decentralization qualitatively or measure only centralized performance; Madapati and Pradhan (2025) is an exception but uses an off-chain pipeline; Rao et al. (2024) provide a scalability review but no cross-paradigm comparison	Direct comparative benchmark against an Express.js + SQLite baseline (Section 7.1.8) plus HHI calculation (Section 7.3.3) make the trade-off numerical.
G6	Regulatory uncertainty for tokenized rights (EU AI Act Aug 2025; MiCA full enforcement Jul 2026; EP report A10-0019/2026)	European Securities and Markets Authority (2025); Guadamuz (2025)	Cryptographic provenance, immutable records, and automated royalty distribution align with emerging compliance infrastructure. MiCA classification of subscription tokens remains an open legal question.

Continued on next page

Table 2.1 continued from previous page

ID	Gap	Supporting literature	CCRMS contribution
G7	No existing system provides first-class native support for subscription, pay-per-view, and permanent ownership as three access modes within a single unified rights primitive. Story Protocol’s PIL can approximate these commercial outcomes with separate license-token configurations; it does not treat renewal and expiry as native state transitions of a single content record.	Patreon (subscription only); OpenSea (ownership only); pay-per-view absent from most blockchain systems; Story Protocol (flexible licensing primitive, not unified rights); Shah (2025) addresses royalty automation as a single feature	Unified rights token: one Content record, three maps, one ordered <code>check_access</code> , and renewal/expiry as first-class storage transitions, not as configurations of a generic licensing primitive. We do not claim that PIL cannot encode similar commercial outcomes.
G8	ink! discontinuation (January 2026) leaves Polkadot without a viable institutionally supported smart contract language; alternatives (revive, raw pallet-revive, wrevive) each carry trade-offs	Appendix A.9	Pallet-first architecture demonstrates that substantial functionality can be delivered without a smart contract language. The gap remains for systems whose business logic is better expressed in contracts.

We fully address gaps G1 and G7; we address G6 and G8 only partially because they depend on factors outside the prototype’s control. The collective contribution is that a single working prototype demonstrates feasibility for a category of systems that the

literature has addressed only partially.

2.5 | Summary

We surveyed the academic and industry literature on blockchain-based content management, cross-chain interoperability, and decentralized monetization. The foundational primitives are in place; the market for decentralized content monetization is real and expanding rapidly; and existing methodologies address the issue only partially. CCRMS bridges the eight identified gaps by deploying a unified rights record on a Polkadot parachain.

Research Questions and Objectives

This chapter presents the research questions and objectives that underpin our study. We formulated them during Phase 1 and refined them in Phase 2 following the gap analysis of Chapter 2.

3.1 | Primary Research Question

The primary research question of our work is:

How can a shared-security, multi-chain framework built on Polkadot's Cross-Consensus Messaging (XCM) and FRAME pallet primitives deliver a unified rights record that natively supports recurring subscriptions, pay-per-view micro-transactions, and permanent ownership transfers across heterogeneous blockchain networks, and what levels of transaction finality, cost efficiency, and creator revenue retention can such a system achieve?

The question has two components: **design feasibility** (whether such a system can be built using the available Polkadot SDK primitives) and **measurement** (what operational properties it exhibits once built). We treat them as mutually constraining; a design that is feasible to build yet performs poorly is not viable, nor is one that performs well in benchmarks but cannot be implemented in practice. The primary concern is framed as an **achievability** question rather than a hypothesis test, consistent with the measurement-focused evaluation philosophy established in Section 4.1.4.

3.2 | Sub-Questions

We decompose the primary research question into five sub-questions. Table 3.1 presents each sub-question, its motivation, and the literature gaps it addresses.

Table 3.1. Research Sub-Questions

ID	Question	Gaps	Motivation
SQ1	To what extent can XCM carry recurring subscription renewal messages and rich rights metadata across Polkadot parachains and external networks, and what atomic success rates and cross-chain finality times are achievable?	G2, G3	Existing cross-chain protocols lack native recurring payments and strip rights metadata. SQ1 asks whether XCM v3 primitives can approximate subscription renewal behavior without custom extensions; the XCM v5 <code>Schedule</code> instruction was specified but not production-ready during the implementation window.
SQ2	What design patterns can minimize access verification latency while maintaining decentralization, and how do the resulting metrics compare across varying validator configurations?	G5	The decentralization-latency trade-off is poorly characterized quantitatively. SQ2 asks for a direct measurement on a working Polkadot parachain with explicit numerical comparisons against centralized alternatives.
SQ3	Can a single on-chain rights object simultaneously enforce subscription auto-renewal, pay-per-view consumption limits, and permanent ownership with automated royalty distribution across chains, and what are the resulting creator cost savings?	G7	No existing system offers all three monetization models as first-class access modes of a single unified primitive. Story Protocol's PIL can be configured to approximate them but requires separate license-token types and does not natively model subscription renewal. SQ3 asks whether a unified primitive can be built and whether its economics justify the effort.

Continued on next page

Table 3.1 continued from previous page

ID	Question	Gaps	Motivation
SQ4	What authorization model is appropriate for cross-chain rights operations that decouple the payer from the beneficiary, and how does it perform under adversarial conditions?	(added Phase 5)	Added in response to a finding from the security audit that cross-chain extrinsics (which decouple the transaction payer from the rights beneficiary) introduce authorization vulnerabilities absent in single-chain calls (Section 7.3.1, Finding B). SQ4 formalizes this concern and asks which authorization model eliminates it.
SQ5	Under realistic creator audience distributions, does the CCRMS model produce demonstrably better creator outcomes than centralized, bridge-based, or marketplace alternatives, and under what conditions?	(added Phase 5)	Added during the Monte Carlo simulation. SQ5 clarifies that the economic question is conditional (“yes, for creators with these characteristics”) rather than a simple yes-or-no.

3.3 | Research Objectives

We pursue the five sub-questions through six research objectives. Table 3.2 summarizes each; Appendix B provides the full statements, deliverables, and revision notes.

Table 3.2. Research Objectives Summary

Objective	Focus	Sub-questions
RO1	Design and prototype XCM extensions for recurring and metadata-rich rights transfers	SQ1, SQ2
RO2	Implement a unified rights record (originally ink!, revised to FRAME pallet) with access-verification latency measurement	SQ2, SQ3

Continued on next page

Table 3.2 continued from previous page

Objective	Focus	Sub-questions
RO3	Merge subscription, PPV, and purchase logic into one composable rights pallet	SQ3, SQ4
RO4	Benchmark scalability, interoperability, and economic efficiency against centralized and bridge-based alternatives	SQ2, SQ5
RO5	Incorporate optional zero-knowledge proof hooks for selective regulatory disclosure (restated as future work in Section 9.3)	Broader feasibility
RO6	Quantify creator savings via Monte Carlo simulation and comparative cost analysis	SQ5

3.4 | Mapping Questions to Objectives, Methods, and Chapters

Table 3.3 maps each sub-question to its research objectives, evaluation methods, and the sections where results are reported.

Table 3.3. Mapping of Sub-Questions to Objectives, Methods, and Chapters

Sub-Question	Primary Objective	Supporting Objective(s)	Primary Method	Reported In
SQ1 (XCM rights operations)	RO1	RO3	XCM latency benchmark; end-to-end XCM test	Section 5.2; Section 7.1.3; Section 7.1.7
SQ2 (Latency vs decentralization)	RO2, RO4	–	Throughput and latency benchmark; HHI calculation	Section 7.1.3; Section 7.3.3

Continued on next page

Table 3.3 continued from previous page

Sub-Question	Primary Objective	Supporting Objective(s)	Primary Method	Reported In
SQ3 (Unified rights token)	RO3	RO2, RO6	Unit tests; royalty propagation tests; Monte Carlo simulation	Section 5.1; Section 7.2
SQ4 (Security and authorization)	(added Phase 5)	RO3	Security audit; authorization unit tests	Section 5.3; Section 7.3.1
SQ5 (Economic viability)	RO6	RO4	Monte Carlo simulation; comparative benchmark	Section 7.2; Section 7.1.8

SQ4 and SQ5 were added in Phase 5 (no direct Phase 2 objective link); RO5 was not implemented and is deferred to future work in Chapter 9.

3.5 | Summary

The five sub-questions and six research objectives are grounded in the gaps identified in Chapter 2 and mapped to evaluation methods and reporting chapters in Table 3.1. SQ4 (security) and SQ5 (economic viability) were added in Phase 5, consistent with the DSR principle that the search process itself may modify the initial framing.

Methodology and System Design

This chapter outlines the research methodology we used to develop CCRMS and the resulting system design. Section 4.1 presents the methodology, including design science framing, iterative prototyping, data collection, and evaluation philosophy. Section 4.2 describes the system design at the conceptual level; Chapter 5 covers implementation details.

4.1 | Research Methodology

4.1.1 | Design Science Research Framing

CCRMS is fundamentally a systems-building project with a primary artifact: a working software prototype. The most appropriate methodological framework is therefore **design science research (DSR)**, as defined by Hevner et al. (2004). Our dissertation aligns with Hevner’s seven DSR guidelines as follows: (1) *design as an artifact*: the CCRMS parachain and seven implementation specifications; (2) *problem relevance*: the commercial and academic gaps identified in Chapter 2; (3) *design evaluation*: eleven KPIs covered in Chapters 6–7; (4) *research contributions*: the artifact, the design knowledge captured in the specifications, and the empirical measurements; (5) *research rigor*: established Polkadot SDK primitives, standard Rust testing, and standard Monte Carlo methodology; (6) *design as a search process*: nine significant pivots documented in Section 5.3; (7) *communication of research*: this dissertation and an open-source code release.

DSR is particularly appropriate here because it treats both the artifact and the design knowledge it produces as coequal contributions.

4.1.2 | Iterative Prototyping

Within the DSR framework, the development process follows an **iterative prototyping** model (build, test, measure, refine). We chose iteration deliberately: the Polkadot SDK monorepo releases stable versions approximately quarterly, making a waterfall approach prone to conclusions based on an outdated toolchain. The architectural intricacies of a cross-chain system mean that certain design decisions can be thoroughly assessed only once the system is operational. In practice, this iterative methodology led to nine substantial architectural pivots during Phase 4, documented as challenges in Section 5.3. We executed the work across six four-week phases (Appendix F); references to “Phase N” throughout the dissertation denote work produced in that phase.

4.1.3 | Data Collection Methods

Our evaluation uses a mixed-methods approach: quantitative measurements include unit tests, throughput and latency benchmarks, storage growth, cross-chain XCM latency, reliability tests, a centralized comparative benchmark, and a Monte Carlo simulation of creator revenue. Qualitative inputs are drawn from a literature synthesis. Chapter 6 describes each method. All quantitative outputs are systematically stored as machine-readable JSON files in the `scripts/perf/results/` directory to ensure reproducibility.

4.1.4 | Evaluation Philosophy: Measurement-Focused, Not Target-Driven

We adopt a **measurement-focused philosophy**: the goal is to document and analyze the system’s performance, efficiency, security, and economic viability rather than to verify that it meets predetermined targets. This measurement-centric approach preserves academic rigor regardless of whether specific results meet hypothetical benchmarks, treats unexpected results as valuable rather than embarrassing, encourages truthful reporting, and establishes a baseline for future researchers.

Each of the eleven KPIs in Section 6.5 has a target derived from the Phase 2 methodology, but we treat those targets as reference points rather than pass/fail criteria. The outcome column in Table 7.15 is therefore headed “Target met?” with Yes/No values rather than Pass/Fail. In practice, all eleven met their targets; the methodology would have remained valid even if some had not.

4.1.5 | Methodology Pivots

The implementation required nine pivots from the Phase 4 plan — substituting Zombienet for Kusama, `pallet-nfts` for RMRK 2.0, and an `on_initialize` block hook for scheduled XCM — each documented as a challenge narrative in Section 5.3 and Appendix A. The broader methodological lesson is that architectural decisions emerging from these pivots constitute research outcomes coequal with the empirical measurements.

4.2 | System Design

This section outlines the conceptual design. Section 5.1 provides the implementation details.

4.2.1 | The Unified Rights Record Concept

This subsection addresses SQ3: it demonstrates that subscription, pay-per-view, and permanent ownership can serve as first-class access modes for a single on-chain record, rather than as separate platform categories.

The principal intellectual contribution of CCRMS is the **unified rights record**: a single on-chain asset that can represent three distinct modes of content access (an active subscription, a remaining pay-per-view balance, or permanent ownership) for a single piece of content from a single creator, all governed by a single set of rules.

The unified token directly closes gap G7 (Table 2.1): a single content registration supports all three access modes concurrently, resolved by a unified priority check.

4.2.2 | The Three Monetization Models

The academic design challenge is to encode three semantically distinct access modes—time-bounded, counter-based, and indefinite—in a single on-chain record without cross-contamination of state across modes. The literature identifies subscription, pay-per-view, and permanent ownership as the canonical creator-economy requirements (Ciriello et al., 2023; Mendoza-Tello et al., 2024), but addresses each in isolation; CCRMS treats their unification into a single primitive as the central design problem.

The architecture supports three models, each motivated by a different gap in the existing landscape:

- **Subscription:** time-limited access valid for a configurable number of blocks per content item, with optional automatic renewal triggered by an on-chain block hook.

- **Pay-per-view (PPV):** prepaid access to a specified number of views of a content item is available. Each `consume_view` operation decrements the counter; additional purchases increase the count rather than overwrite it. Conceptualizing PPV as a counter rather than a sequence of individual view transactions avoids the micro-transaction problem that has historically impeded on-chain PPV models (Chegenizadeh et al., 2024).
- **Permanent ownership:** unrestricted and transferable access. Ownership is prioritized during the access verification process. Transfers follow a mint-burn-mint pattern rather than a direct NFT transfer, preserving provenance and attribution consistency.

A single content item can offer all three models concurrently, each with independently configured pricing. The `check_access` function resolves rights in priority order (Ownership > Subscription > PPV).

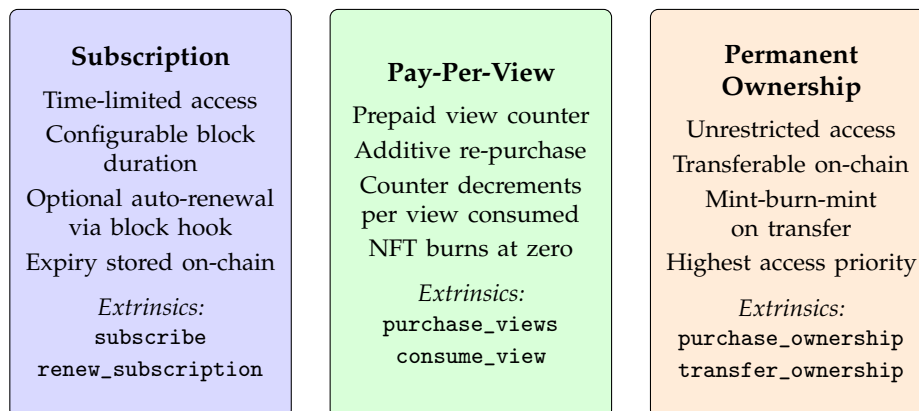


Figure 4.1. The three monetization models natively supported by the unified rights record (addresses SQ3). A single content registration may offer all three models concurrently; `check_access` resolves in priority order: Ownership > Subscription > PPV.

4.2.3 | NFT-Based Rights Tokenization

NFT-based tokenization provides the technical foundation for SQ3: standard `pallet-nfts` primitives can encode RMRK 2.0 composability semantics for a unified token, without relying on unmaintained pallets.

CCRMS encodes content rights as non-fungible tokens using a parent-child nesting model built on `pallet-nfts`. The decision to use NFTs rather than a bespoke storage configuration was motivated by their widespread acceptance as a standard, compatibility

with existing digital wallets, independent transferability (which facilitates a secondary market for ownership rights), and the clear separation of the content item’s identity (the parent) from user-specific rights derived from it (the children). The mint-burn-mint transfer pattern for the permanent ownership mode aligns with the approach proposed by Dondjio and Kazamias (2024), who demonstrate that re-minting on transfer preserves on-chain provenance without the ambiguity of direct NFT reassignment.

The conceptual model: each content registration is an NFT collection comprising one parent Content NFT and zero or more child rights NFTs, each tagged with a `rights_type` attribute (Subscription, PPV, or Ownership) and owned by the respective user. The parent-child structure serves three purposes: establishing identity, tokenizing individual rights, and enabling on-chain auditability of the rights-to-content relationship.

The nesting pattern is inspired by RMRK 2.0, the composable NFT specification for Polkadot; however, it is implemented directly on top of `pallet-nfts` rather than relying on the `rmrk-substrate` pallets, which are locked to SDK 0.9.36. Appendix A.1 details this transition. The outcome shows that RMRK-style composability can be achieved using only standard FRAME primitives.

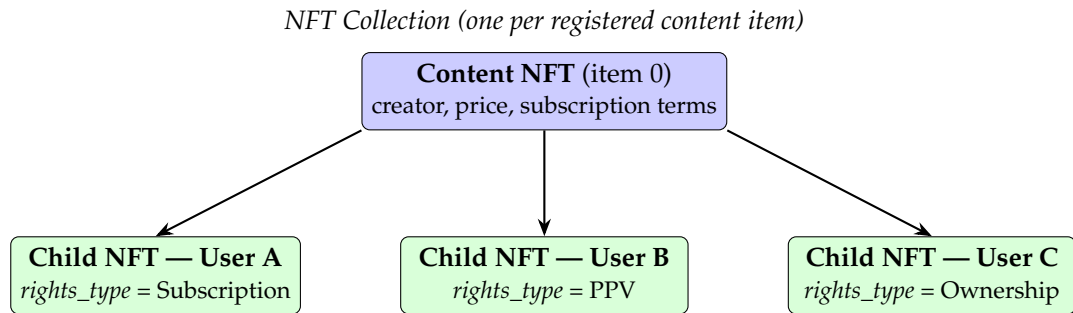


Figure 4.2. Parent-child NFT nesting model. Each content registration creates one NFT collection containing a parent Content NFT (the canonical on-chain identity of the content) and zero or more child rights NFTs, each owned by an individual user and tagged with their access mode.

One significant constraint is that the `Children` storage entry is a `BoundedVec` (a fixed-capacity collection type in FRAME that enforces an explicit upper bound at the type level), limiting each content item to fifty child NFTs. Because each first-time purchase mints one for the buyer and expired subscriptions never release theirs, this caps a content item at fifty subscribers and owners over its lifetime. The bound is not intrinsic to the design; holders are already recorded in the three rights maps (Appendix E.4). The examination of this constraint is detailed in Appendix A.8, and mitigation is described in Appendix E.4.

4.2.4 | Cross-Chain Communication Model

This model addresses SQ1 (XCM rights operations across chains) and SQ4 (payer–beneficiary authorization): the two properties that make cross-chain rights management nontrivial are the decoupling of payment from rights receipt and the trustless verification of remote rights state.

Cross-chain operation is the mechanism by which the unified rights record model enables portable digital ownership: a user who acquires rights on one blockchain should be able to use them on another without losing access or repurchasing. The model has two conceptual properties.

Payer–beneficiary decoupling. Each cross-chain rights operation distinguishes between the funding account and the rights recipient account. In a local call, the payer and the beneficiary are the same (the caller); in a cross-chain call, the dispatch origin is the sovereign account of the remote parachain, and the beneficiary is a separate user account. This configuration allows a remote parachain to acquire rights on behalf of its users without requiring those users to be present on CCRMS.

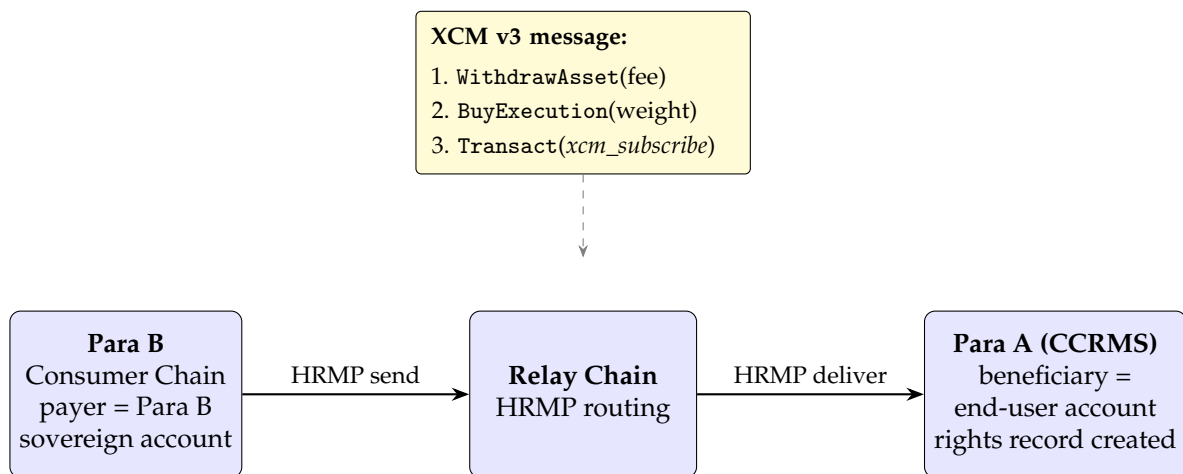


Figure 4.3. XCM cross-chain rights operation (addresses SQ1 and SQ4). The three-instruction sequence dispatches an `xcm_*` extrinsic on the CCRM parachain, decoupling the paying account (Para B sovereign account) from the rights beneficiary (the end user). The same pattern applies to all five `xcm_*` extrinsics.

Trustless verification of remote rights. A second pallet, `pallet-rights-verifier`, provides Merkle proof verification of the CCRM rights state for other parachains. A consuming chain can verify, without relying on any oracle or relayer, that a specific account holds an active subscription by submitting a storage proof against a known CCRM state root. This approach upholds the principle that CCRM rights are **canonical on**

the CCRMS chain: other chains authenticate cryptographically rather than maintaining replicas.

The model also extends beyond Polkadot via **Snowbridge**, a trust-minimized bridge connecting Polkadot and Ethereum. We configure the runtime to accept messages from Ethereum using the standard `LocationToAccountId` converters. At this stage, our architectural choice was to ensure **forward compatibility with Ethereum bridging without relying on it**. Section 5.2 provides a comprehensive account of the cross-chain integration.

4.2.5 | Pallet-First Architecture

The pallet-first decision directly shapes the answers to SQ2 (latency versus decentralization) and SQ3 (unified primitive): placing business logic in a FRAME pallet rather than in smart contracts reduces execution overhead and eliminates the chain-extension boundary that would otherwise constrain write operations.

The most significant architectural decision was to centralize the business logic within a **FRAME pallet** rather than in smart contracts. This decision was made during Phase 4 for three reasons: the operational maturity gap between FRAME pallets and `pallet-revive`; the unavailability of the chain extension mechanism, which the original contract-first approach relied on for write operations; and a pallet-first design that uses standard Polkadot wallets without EVM-style adaptation. Appendix A.2 documents this decision; the January 2026 ink! discontinuation announcement (Appendix A.9) confirmed the engineering rationale.

4.2.6 | Layered Architecture

The layered architecture determines the answers to SQ1 (cross-chain reach) and SQ2 (the latency–decentralization trade-off): each layer can evolve independently, so XCM protocol upgrades and relay-chain migration to JAM do not require changes to the Layer 3 business logic.

The system is organized into six distinct, independently replaceable layers: (1) the **Polkadot relay chain** provides shared security and enables HRMP message passing; (2) the **parachain runtime** composes the CCRMS FRAME pallets, configures the XCM executor, and includes a custom EVM precompile; (3) the **business logic** layer comprises `pallet-content-rights` (17 extrinsics, 10 storage items) and `pallet-rights-verifier`; (4) the **NFT layer** uses the standard `pallet-nfts`, with parent–child nesting enabled by `pallet-content-rights`; (5) the **smart contract / precompile layer** provides

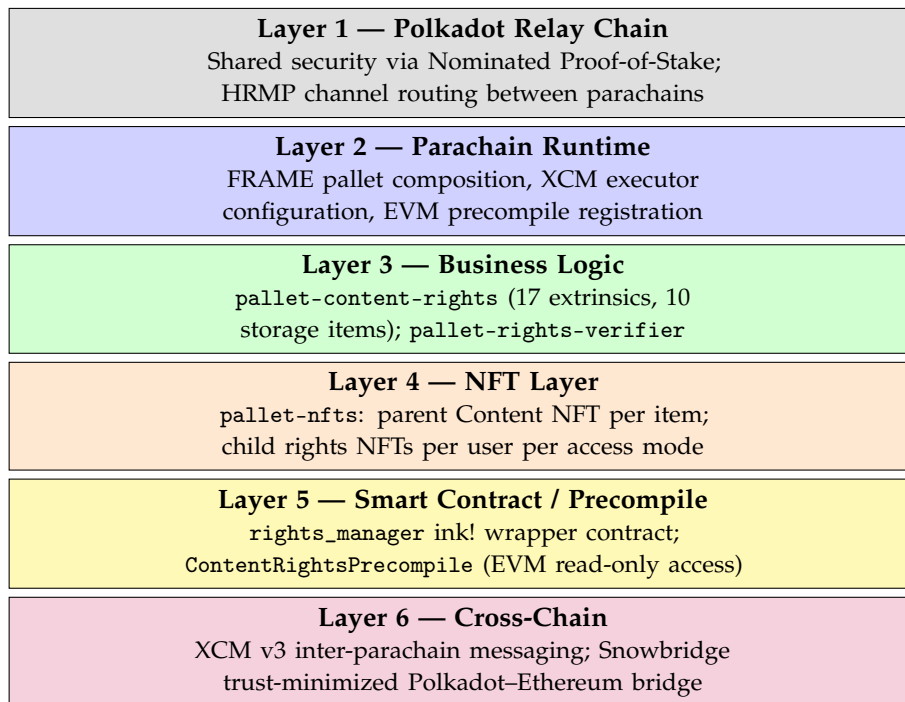


Figure 4.4. CCRMS six-layer architecture (addresses SQ1 and SQ2). Each layer is independently replaceable: Layer 5 (smart contracts) may be removed, Layer 6 (XCM) may upgrade from v3 to v5, and Layer 1 (relay chain) may migrate to JAM, without modifying the business logic in Layer 3.

EVM- and ink!-compatible access through the `rights_manager` contract and the `ContentRightsPrecompile`, both of which are non-load-bearing wrappers; (6) the **cross-chain layer** manages XCM v3 messaging and the Snowbridge integration. Independence among layers is a fundamental design objective: the smart contract layer may be removed or replaced, the XCM protocol may evolve from v3 to v5, and the relay chain may transition to JAM, all without affecting the pallet code.

4.3 | Summary

This chapter framed CCRMS within the design science research paradigm and presented the six conceptual design decisions that govern implementation: the unified rights record (closing gap G7), the three monetization models, NFT-based tokenization via `pallet-nfts`, the cross-chain payer-beneficiary model, the pallet-first architecture, and the six-layer system structure. Chapter 5 covers implementation details.

Implementation

This chapter describes what was built and how key decisions played out in practice. The development environment, toolchain, and workspace layout are documented in Appendix C. Section 5.1 summarizes the pallet implementation; Section 5.2 covers cross-chain integration; Section 5.3 documents the nine challenges encountered and their resolutions.

5.1 | Pallet Implementation and Smart Contract Layer

Figure 5.1 shows the six implementation components and their relationships.

The core of CCRMS is `pallet-content-rights`, a FRAME pallet that implements all business logic: content registration, the three monetization models (subscription, pay-per-view, and permanent ownership), on-chain royalty distribution to up to ten collaborators, automated subscription renewal via an `on_initialize` hook, and cross-chain variants of every financial operation. It exposes 17 dispatchable extrinsics, 10 storage items, and 22 event variants; the full reference tables are in Appendix C. Because `check_access` is invoked via XCM from remote parachains and each call is recorded on-chain for auditability, it is implemented as a signed extrinsic rather than a free runtime API. Content rights are represented as NFT collections using a parent-child nesting model built on `pallet-nfts`: a parent Content NFT is the canonical on-chain identity for each piece of content, and each user's access rights are a child NFT tagged with a `rights_type` attribute (Subscription, PPV, or Ownership). The smart contract and precompile layers are stateless wrappers; all authoritative state resides in the pallet (Appendices A.2 and A.9).

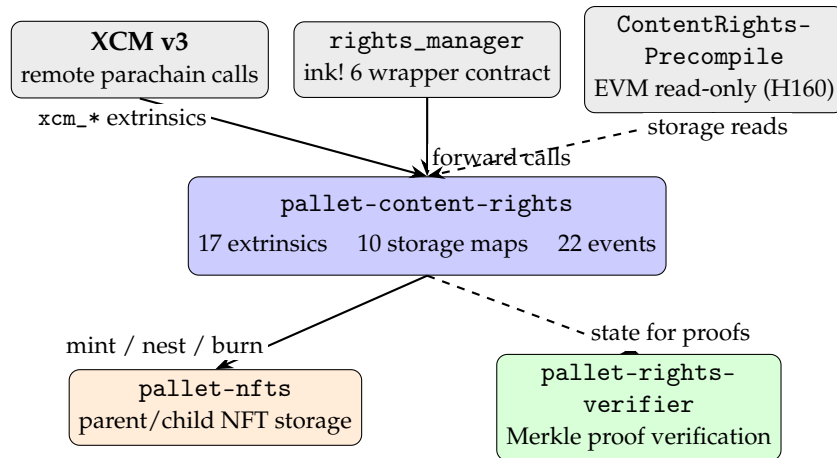


Figure 5.1. CCRMS implementation-level component diagram. Solid arrows indicate active extrinsic dispatch; dashed arrows indicate read-only storage access. `pallet-content-rights` is the sole authoritative state layer; the ink! contract and EVM precompile are stateless wrappers that forward calls or read storage, respectively. XCM delivers cross-chain calls via HRMP. `pallet-rights-verifier` enables remote parachains to validate rights state via Merkle storage proofs, without oracles or relayers.

5.2 | Cross-Chain Integration

The core challenge in cross-chain integration is enabling remote parachains to acquire and verify content rights without a shared trust anchor or custodian. Two sub-problems follow: (1) how to dispatch authenticated, fee-paying rights operations across HRMP in a single atomic message; and (2) how to extend that reach to Ethereum without custodial bridges. The key lesson from both is that the CCRMS runtime’s XCM configuration and payer-beneficiary decoupling are sufficient; no custom XCM extensions were required.

5.2.1 | XCM Message Flows

Cross-chain operations use Polkadot XCM v3 to enable remote parachains to acquire content rights. The pallet exposes five XCM extrinsics: `xcm_subscribe`, `xcm_renew_subscription`, `xcm_purchase_views`, `xcm_purchase_ownership`, and `xcm_transfer_ownership` (callable locally only after Finding B remediation; see Appendix D). Each separates the dispatch origin (**payer**) from the **beneficiary** of the rights record: when a remote parachain sends a Transact message, the origin is the remote parachain’s sovereign account, but the rights record is issued to a distinct beneficiary account representing the end user on the remote chain.

The runtime’s XCM configuration follows the standard Cumulus pattern,

with one customization: the `LocationToAccountId` chain adds `HashedDescription<_, DescribeFamily<DescribeAllTerminal>>` and `GlobalConsensusParachainConvertsFor<_, _>` so the runtime can interpret Ethereum locations of the form `GlobalConsensus(Ethereum) → AccountKey20(0x...)`. This is required for Snowbridge (Section 5.2.2); the full Ethereum-to-CCRMS flow is configured but has not been end-to-end tested.

Each XCM rights operation uses the three-instruction sequence `WithdrawAsset`, `BuyExecution`, and `Transact`. This is sufficient because rights operations are stateful side effects, not asset transfers (Appendix C shows the full message structure). The endianness and fee-sizing findings (A.5, A.6 in Table 5.1) are the two most operationally significant surprises in the XCM integration.

We measured end-to-end XCM latency at approximately 100 seconds of wall-clock time (a 3–5-block delta on the CCRMS chain) for `xcm_subscribe`, `xcm_purchase_views`, and `xcm_purchase_ownership` on the local Zombienet topology, full measurements are reported in Chapter 7. The block delta underestimates the true elapsed time: the relay-chain hop and HRMP processing pipeline add ~ 70 seconds of wall-clock overhead that is not reflected in the CCRMS block count. Polkadot 2.0 features (Async Backing, Agile Coretime, Elastic Scaling) would substantially reduce the latency floor in production.

A secondary pallet, `pallet-rights-verifier`, provides trustless verification of CCRMS rights status for other parachains via Merkle storage proofs. A remote parachain can verify an active subscription without oracles or relayers by submitting a proof against a known CCRMS state root, which the verifier replays against the Substrate trie. Three extrinsics (`verify_ownership`, `verify_subscription`, `verify_view_pack`) construct a storage key, validate the proof, and emit a result event. Proof validation costs roughly 100 million `ref_time` per call and avoids any network round-trip.

5.2.2 | Snowbridge Integration with Ethereum

We extended the cross-chain architecture to Ethereum via Snowbridge, a trust-minimized bridge that uses light-client verification on both sides, eliminating custodians and relayer trust. During Phase 4 we bridged two 1-ETH transactions from a local Ethereum chain to a local AssetHub parachain using the Snowbridge pipeline, demonstrating that the CCRMS runtime can accept bridged Ethereum messages with the same XCM configuration as sibling-parachain messages (via `HashedDescription` and `GlobalConsensusParachainConvertsFor` in `LocationToAccountId`).

The topology spans four parachains and a local Ethereum network: CCRMS (parachain 100); Bridge Hub (parachain 1013), which hosts the inbound queue and the Ethereum beacon light client; AssetHub (parachain 1000), which holds the foreign-asset

reserve; and the Rococo relay chain. Four HRMP channels link Bridge Hub ↔ AssetHub and AssetHub ↔ CCRMS, carrying bridge messages and enabling future Polkadot–Ethereum interoperability.

The Ethereum side runs Geth v1.17.1 (chain ID 11155111) as the execution layer, Lodestar v1.35.0 as the beacon node, and the 16-contract Snowbridge Gateway suite was deployed via Forge (a Solidity contract deployment tool from the Foundry framework). After initializing the beacon light client, two relayer processes run: a beacon relay that forwards Lodestar finality events to the `EthereumBeaconClient` pallet, and an execution relay that forwards Gateway events from Geth to the `EthereumInboundQueue` pallet.

The integration revealed three persistent issues, documented in Appendix A: a Lodestar version-pinning requirement that forces source compilation; a configuration bug in the fork version that produced invalid Merkle proofs; and a beacon state cache timing issue that required reordering the setup sequence.

The final step, using bridged Ether on AssetHub to pay for a CCRMS rights operation, is conceptually straightforward but was not implemented in the prototype. The CCRMS XCM side is already prepared; the remaining task is to configure `pallet-content-rights::PaymentCurrency` to accept both foreign Ether and the native token. This is listed as future work in Chapter 9.

5.3 | Challenges and Solutions

The CCRMS implementation encountered nine technical issues that shaped the final architecture and security posture. Table 5.1 summarizes each issue, our response, and the key lesson; Appendix A provides full details.

Table 5.1. Implementation Challenges Summary

#	Challenge	Response	Key Lesson
A.1	RMRK 2.0 pallets frozen on SDK 0.9.36, incompatible with current monorepo	Replicated RMRK nesting semantics on <code>pallet-nfts</code> directly	Standards reimplementations are preferable when reference implementation is unmaintained

Continued on next page

Table 5.1 continued from previous page

#	Challenge	Response	Key Lesson
A.2	Contract-first design blocked: chain extensions non-functional for writes, debugging complexity	Inverted to pallet-first; retained ink! contract as thin wrapper	Pallet-first proved resilient to ink! discontinuation (A.9)
A.3	Snowbridge beacon relay generated proofs at wrong gindex (54 vs. 86)	forkVersions field means activation epoch, not version hex; changed to {"deneb": 0, "electra": 0}	Cryptographic systems fail silently on configuration errors
A.4	Lodestar v1.41.0 ignored LODESTAR_PRESET=mainnet in dev mode (32 vs. 512 validators)	Pinned Lodestar v1.35.0, built from source	Light clients impose strict counterpart requirements; bridge toolchain compatibility is fragile
A.5	XCM tests: funds debited on source chain but InsufficientPayment on CCRMS	polkadot.js toHex() used big-endian; CCRMS expects little-endian; fixed with toHex(true)	Endianness bugs produce valid-looking but wrong addresses
A.6	XCM BuyExecution failed at 1B tokens; required ~75–100B	proof_size dominates BlockRatioFee, not ref_time	Polkadot fee modeling is driven by proof size, not computation
A.7	xcm_transfer_ownership had no check that caller == from (Medium severity)	Added ensure!(authorizer == from); unit test confirms block	Cross-chain extrinsics accepting identity parameters need explicit authorization
A.8	MaxChildrenReached at 50 subscribers per content item	Documented as known limitation; benchmark scripts rotate content items	Bounded FRAME collections trade benchmarkability for capacity limits

Continued on next page

Table 5.1 continued from previous page

#	Challenge	Response	Key Lesson
A.9	ink! development discontinued Jan 2026	95% of logic in pallet; contract layer is non-load-bearing; migration deferred	Minimize dependency on non-essential components for ecosystem resilience

5.4 | Summary

Cross-chain rights operations are delivered via XCM v3 with payer–beneficiary decoupling, and the Snowbridge integration configured the path for Ethereum bridging, though the Ethereum-to-CCRMS flow has not been end-to-end tested. Nine challenges shaped the final architecture; the two most consequential—the RMRK-to-pallet-nfts pivot and the contract-first-to-pallet-first pivot—are examined in greater detail in Chapter 8. Full implementation details are provided in Appendix C.

Evaluation and Testing

This chapter outlines the evaluation methodology used to assess CCRMS against the research questions in Chapter 3. It focuses on **how** the system was evaluated; the results are presented in Chapter 7. This chapter covers the evaluation approach, the four-layer testing architecture, the eight test scenarios, the 11 KPIs, the testing infrastructure, and the validity considerations that inform our interpretation of the findings.

6.1 | Evaluation Approach

Our evaluation methodology spans unit testing, integration testing, performance benchmarking, security assessment, comparative analysis against a centralized baseline, and Monte Carlo simulation, covering the technical, economic, and security domains of the research questions. Table 6.1 maps each evaluation activity to the specific research sub-questions it addresses.

Table 6.1. Mapping of Evaluation Activities to Research Sub-Questions

Research Question	Primary evaluation activity	Supporting activities
SQ1 (XCM cross-chain rights operations and finality)	XCM latency benchmark, end-to-end XCM test	Unit tests for xcm_* extrinsics
SQ2 (Latency vs decentralization trade-offs)	Throughput and latency benchmark, HHI calculation	Comparative analysis vs centralized

Continued on next page

Table 6.1 continued from previous page

Research Question	Primary evaluation activity	Supporting activities
SQ3 (Unified rights token with royalty distribution)	Unit tests, royalty propagation tests, comparative benchmark	Monte Carlo creator revenue simulation
SQ4 (Security and authorization)	Security audit, authorization unit tests	Reliability test
SQ5 (Economic viability)	Monte Carlo creator revenue simulation	Literature comparison

6.2 | Testing Layers

We organize testing into four layers, each executed at the lowest applicable level of abstraction. Figure 6.1 summarizes the stack; each layer is described below.

6.3 | Performance Testing Framework

Seven automated scripts in `scripts/perf/` cover various aspects of system behavior and each produce machine-readable JSON output. Table 6.2 summarizes the benchmark categories.

Table 6.2. Benchmark Scripts

Category	Script	What it measures
Throughput	<code>local-throughput.mjs</code>	TPS across 11 extrinsic types at batch sizes 1, 10, 20, 50, 100
Stress	<code>stress-test.mjs</code>	Sustained TPS from 200 concurrent accounts over 180 seconds
XCM latency	<code>xcm-latency.mjs</code>	Block delta between send (Para B) and event arrival (Para A)
Block weight	<code>block-utilization.mjs</code>	Percentage of <code>ref_time</code> and <code>proof_size</code> consumed at batch sizes 50–300

Continued on next page

Table 6.2 continued from previous page

Category	Script	What it measures
Storage	storage-growth.mjs	SCALE-encoded byte size per content item, subscriber, and view pack
Resource monitoring	resource-monitor.mjs	CPU, memory, and disk usage via Prometheus scraping during load
Reliability	reliability-test.mjs	Baseline uptime, MTTR after SIGTERM, state persistence, recovery uptime
Comparative	centralized-drm-benchmark/	Same six operations on Express.js + SQL.js under identical conditions

A separate Monte Carlo simulation (`monte-carlo-revenue.mjs`) models 1,000 creators over 10,000 iterations across four platform configurations (centralized 30–45% fees, Web3 marketplace 5–10%, bridge-based 8–15%, CCRMS 1–5%), incorporating algorithmic discovery and monthly churn. This simulation provides the primary evidence for research question SQ5. We document the methodology in `scripts/perf/results/monte-carlo-findings.md`.

All scripts are reproducible: each is an independent Node.js program with explicit parameters and deterministic output paths, and each is archived as timestamped JSON in `scripts/perf/results/`.

6.4 | Test Scenarios

We designed eight test scenarios to cover the full range of evaluation activities. Table 6.3 summarizes each scenario, and the corresponding scripts are located in `scripts/perf/`.

Table 6.3. Test Scenarios

#	Scenario	Description	Target
1	Throughput and latency	Submit concurrent batches (1, 10, 20, 50, 100) for each of the 11 local extrinsics; measure submission-to-inclusion time.	>10 TPS sustained; ~6 s inclusion latency

Continued on next page

Table 6.3 continued from previous page

#	Scenario	Description	Target
2	Block weight saturation	Submit progressively larger <code>register_content</code> batches (50–300); query <code>system.blockWeight</code> at inclusion.	Block weight <75% of budget at max batch
3	Storage growth	Register content, subscribers, and view packs in incremental quantities; measure SCALE-encoded byte size per map.	<500 bytes per content item; linear growth
4	XCM latency	Send <code>xcm_subscribe</code> , <code>xcm_purchase_views</code> , <code>xcm_purchase_ownership</code> from Para B to Para A via the canonical <code>WithdrawAsset</code> → <code>BuyExecution</code> → <code>Transact</code> sequence; measure block delta.	<10 parachain blocks per operation
5	Stress test	200 funded accounts continuously submit <code>register_content</code> for 180 seconds.	Sustained TPS substantially above single-batch throughput
6	Security audit	Static analysis (cargo <code>clippy</code> , cargo audit), manual review of all 17 extrinsics, targeted unit tests for authorization gaps and edge cases.	0 Critical / 0 High vulnerabilities; complete authorization coverage
7	Reliability	Monitor block production for 120 s (baseline), SIGTERM the collator, restart, verify state, monitor for 120 s (recovery).	MTTR <60 s; 100% state persistence
8	Comparative analysis	Re-implement the same six operations in <code>Express.js</code> + <code>SQL.js</code> (in-memory <code>SQLite</code>); run identical benchmarks against both.	Document the performance cost of decentralization

The comparative benchmark is intentionally minimal, using `Express.js` for HTTP and

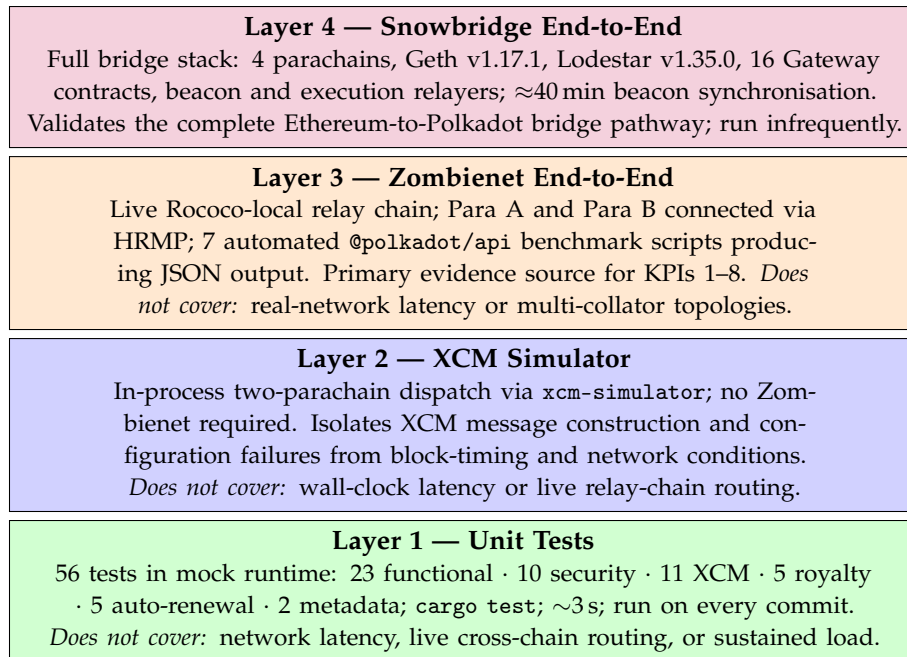


Figure 6.1. Four-layer testing architecture (Layer 1 at base: fastest and most granular, run on every commit; Layer 4 at apex: broadest scope, run infrequently). Each layer tests what lower layers cannot: unit tests isolate individual extrinsic logic; the XCM simulator isolates message-construction failures; Zombienet provides definitive wall-clock performance evidence; Snowbridge validates the complete Ethereum bridge pathway.

SQL.js for in-process SQLite. A more comprehensive, centralized implementation with caching, replication, and distributed databases would perform worse than our baseline but would still be significantly faster than CCRMS. The minimal version serves as a near-optimal reference point for comparison and precludes any implication of manipulation.

6.5 | Metrics and Key Performance Indicators

Our evaluation uses 11 key performance indicators (KPIs) across four domains: technical, economic, security, and reliability. Each KPI includes a description, measurement methodology, originating source (the test scenario or activity that generates it), and a target value. These targets were set as stated design objectives during Phase 2 planning, informed by commercial comparators rather than by published academic benchmarks; they are treated throughout as reference points rather than strict pass/fail criteria. The inclusion latency target of two block intervals reflects the 6-second parachain block time of the test environment; the uptime target of 85% is sized for a single-collator Zombienet

topology, not a production service-level agreement.

Table 6.4. KPI Definitions

#	KPI	Definition	Target	Source
1	Throughput (TPS)	Successful transactions per second under sustained load	>10 TPS	Stress test
2	Inclusion latency	Time from submission to block inclusion	<12 seconds (2 blocks)	Throughput test
3	XCM latency	Block delta for cross-chain operations	<10 blocks	XCM latency test
4	Block utilization	Percentage of block weight consumed at capacity	<75%	Block saturation test
5	Storage efficiency	Bytes per content item and per subscriber	<500 bytes	Storage growth test
6	Uptime	Percentage of expected blocks actually produced	>85%	Reliability test
7	MTTR	Time from crash to resumed block production	<60 seconds	Reliability test
8	State persistence	Data integrity after crash restart	100%	Reliability test
9	Security findings	Critical/High vulnerabilities in dissertation code	0 Critical, 0 High	Security audit
10	Test coverage	Error variants exercised by unit tests	>80%	Test suite
11	Decentralization ratio	Performance cost vs centralized baseline	Documented	Comparative analysis

The target value and measurement source for each KPI are defined in Table 6.4 above. Measured values and outcomes for all 11 KPIs are reported in Chapter 7 (Table 7.15). The security audit identified nine findings (see Table 7.13, Appendix D): two vulnerabilities remediated (B, E), one open denial-of-service path (I), three accepted design properties (A, C, D), and three positive reliability observations (F, G, H). Chapter 7 presents the

full KPI outcomes and discusses each in relation to the research questions; Chapter 8 interprets the findings.

6.6 | Testing Infrastructure

Table 6.5. Test Environment Specifications

Component	Specification
Machine	macOS Tahoe 26.5.2, Apple Silicon
Relay chain	Rococo-local, 2 validators (alice, bob)
Parachain A	Content Rights (parachain 100), 1 collator
Parachain B	Consumer chain (parachain 200), 1 collator
Bridge Hub	Parachain 1013 (Snowbridge tests only)
AssetHub	Parachain 1000 (Snowbridge tests only)
Ethereum	Geth v1.17.1 + Lodestar v1.35.0 (Snowbridge tests only)
Zombienet provider	Native
Block time	~6 seconds (parachain)
Node binary	parachain-template-node (release build)
Polkadot SDK version	stable2512 (December 2025)
Test scripts	scripts/perf/*.mjs (Node.js, @polkadot/api library)
Centralized benchmark	centralized-drm-benchmark/ (Express.js + SQL.js)

The single-machine topology simplifies setup and eliminates reliance on external infrastructure, with implications for interpreting the results discussed in Section 6.7. Benchmark scripts and raw JSON outputs are version-controlled, allowing any reader with access to the Polkadot SDK toolchain to independently re-execute them.

6.7 | Validity and Limitations

Internal validity. We run all benchmarks multiple times and report the average results; maintain a consistent test environment across all runs; apply the same methodology to CCRMS and the centralized baseline; and run the deterministic unit test suite on each commit.

External validity. This is the dimension in which our evaluation is most limited. Four constraints apply: (1) we measure in a local Zombienet environment with two

relay-chain validators and no real network latency, so the figures serve as upper bounds within the test environment rather than predictive metrics for production; (2) the Snow-bridge integration uses a local Ethereum stack rather than a public testnet, avoiding variability in block times, MEV, contention, and beacon chain reorganizations; (3) because of the maturation gap of Polkadot 2.0, the latency figures reflect a pre-2.0 toolchain, and several measurements are expected to improve significantly in a production Polkadot 2.0 environment; (4) our scale is modest, comprising up to 200 concurrent accounts and a few hundred content items, and empirical verification of linear storage growth at production scale has not yet been conducted.

Construct validity. The figure of 26.2 TPS quantifies the successful inclusion rate under sustained load but is insufficient to assess user-perceived performance, which also depends on wallet user experience, mempool latency, and RPC responsiveness. The baseline uptime of 90% assesses single-collator block production and does not account for multi-collator production availability. The Monte Carlo simulation evaluates expected revenue using a probabilistic model; the input parameter sources and assumptions are documented in Appendix G.

Threats to validity. (a) **Selection bias:** the eight test scenarios may not capture production conditions such as high subscriber concurrency on a single content item, large multi-collator topologies, or extended multi-month uptime. (b) **Measurement instrument bias:** any bug in `@polkadot/api` or `system.blockWeight` would propagate into our results; although the library is widely used, it is not bug-free. (c) **Confirmation bias:** the author was responsible for designing, implementing, and evaluating the system; this risk is mitigated by using externally referenced KPI targets and by disclosing all scripts and raw results. (d) **Single-author validation:** no external code review, third-party security audit, or benchmark validation has been conducted; we recognize this as a significant limitation and intend to address it in future work (Appendix E, E.1).

6.8 | Summary

The evaluation uses four testing layers, eight test scenarios, seven automated benchmarks, a Monte Carlo simulation, and 11 KPIs grounded in a measurement-focused philosophy. Section 6.7 outlines the primary validity constraints. Chapter 7 presents the results.

Results

This chapter presents the measured results. Section 7.1 covers technical results, Section 7.2 covers economic results, and Section 7.3 covers security and user-centric results. We defer interpretation to Chapter 8.

7.1 | Technical Results

The technical results address SQ1 (Section 7.1.3), SQ2 (Section 7.1.8), and SQ3 (Sections 7.1.1–7.1.5).

7.1.1 | Throughput

Table 7.1 reports 100 concurrent extrinsics per type against a single collator (6-second blocks). All four non-minting extrinsics achieved 100/100 inclusion in one block (≥ 100 extrinsics per block, ≈ 16.6 TPS). No failure arose from block weight, space, or transaction-pool capacity.

All failures were dispatch errors from the fifty-child bound (Section 4.2.3): `MaxChildrenReached` directly, or cascading `SubscriptionNotFound/ViewPackNotFound` for dependent extrinsics. The benchmark characterizes storage-design capacity, not parachain throughput. Sustained throughput is in Section 7.1.2.

Per-extrinsic TPS is not reported: elapsed time reflects how many blocks the harness waited before observing inclusion (1–4 blocks), not execution cost. Relative costs are in the FRAME weights (Appendix C; hand-estimated, not benchmarked).

Table 7.1. Single-Chain Inclusion Capacity at Batch Size 100

Extrinsic	Succ.	Failed	Blocks	Failure cause
register_content	100	0	1	—
check_access	100	0	1	—
set_royalty_splits	100	0	1	—
query_rights_metadata	100	0	1	—
subscribe	50	50	1	MaxChildrenReached
purchase_ownership	50	50	1	MaxChildrenReached
purchase_views	0	100	0	MaxChildrenReached
renew_subscription	50	50	1	SubscriptionNotFound [†]
enable_auto_renew	50	50	1	SubscriptionNotFound [†]
disable_auto_renew	50	50	1	SubscriptionNotFound [†]
consume_view	22	78	1	ViewPackNotFound [†]

[†] Cascading consequence of the fifty-child bound: the prerequisite subscriptions and view packs could not be created, so the dependent extrinsics had no state to act on.

7.1.2 | Sustained Throughput (Stress Test)

Under sustained load (200 accounts, 180 s), the system achieved 3,875 successful extrinsics at a **96.9% success rate** over the first 142 s, yielding **27.3 TPS** (174.6 transactions per block). The overall failure count is a harness artifact: nonce exhaustion caused the benchmark client to spin thousands of submissions in six seconds at 142–148 s; this was a client limit, not chain capacity.

7.1.3 | Latency

Single-chain inclusion latency was consistently **~6,000 ms** (one parachain block; s.d. <15 ms), confirming atomic same-block inclusion. XCM cross-chain latencies are in Table 7.2 ($n = 3$ per operation).

Table 7.2. XCM Destination-Chain Inclusion Delay by Operation ($n = 3$ per operation)

Operation	Mean Block Delta	Standard Deviation	15-block scan window
xcm_subscribe	3.0 blocks	± 2.0 blocks	~ 99.6 seconds
xcm_purchase_views	5.3 blocks	± 1.5 blocks	~ 100 seconds
xcm_purchase_ownership	5.3 blocks	± 1.5 blocks	~ 100 seconds

The destination event was observed within 15 blocks in all nine runs (median delta 5, ~ 30 – 35 s at 6.67 s/block); the ~ 100 s wall-clock measures producing 15 destination blocks. Async Backing and Agile Coretime (Parity Technologies, 2025) would reduce these delays.

7.1.4 | Block Weight Utilization

Table 7.3. Block Weight Utilization at Increasing Batch Sizes

Batch Size	ref_time Consumed	proof_size Consumed	Block Construction Time
50	2.2%	0.4%	~ 10 ms
100	4.3%	0.7%	~ 21 ms
150	6.5%	1.1%	~ 31 ms
200	8.6%	1.5%	~ 42 ms
250	10.7%	1.8%	~ 52 ms
300	12.9%	2.2%	~ 62 ms

At 300 transactions, block weight accounts for 12.9% of ref_time and 2.2% of proof_size, with a construction time of approximately 62 ms ($\sim 1\%$ of the 6-second block). PoV weight is zero in every hand-estimated weight (Appendix C); this figure reflects reference time only.

7.1.5 | Storage Growth

Table 7.4. Per-Item Storage Costs (SCALE-encoded)

Item Type	Storage Map	Bytes per Item
Content registration	Contents	191
Active subscription	Subscriptions	112
View pack	ViewPacks	111
Ownership record	Ownership	36
Royalty splits (per collaborator)	RoyaltySplits	34
Child NFT nesting record	Children	8 (per child)

Growth was perfectly linear (the 50th item costs the same as the first), confirming $O(1)$ StorageMap complexity. At one million content items and ten million subscribers, projected state is approximately 1.3 GB, within the 500-byte-per-record Phase 2 target.

7.1.6 | Reliability

Table 7.5. Reliability Test Results

Phase	Metric	Value
Baseline	Blocks produced	18 of 20 expected
Baseline	Uptime	90%
Baseline	Mean block time	6.67 seconds
Failure	Time to RPC ready (post-restart)	~15 seconds
Failure	Time to first new block (post-restart)	~20 seconds
Recovery	State integrity	100% preserved
Recovery	Blocks produced	18 of 20 expected
Recovery	Uptime	90% (matches baseline)
Recovery	Mean block time	6.67 seconds (matches baseline)

A 90% block production rate reflects typical single-collator Zombienet behavior (Finding F); the collator did not go down. MTTR was approximately 15 s for RPC and 20 s for the first new block, with 100% state preservation. Recovery metrics matched the baseline exactly.

7.1.7 | Snowbridge End-to-End Bridge

We bridged two 1 ETH transactions from the local Ethereum network to AssetHub via Snowbridge (four parachains, Geth, Lodestar, 16 Gateway contracts, two relayers). Both succeeded; the destination account held 22 ETH on AssetHub (20 pre-minted, 2 bridged). Table 7.6 provides the message pathway.

Table 7.6. Snowbridge End-to-End Message Path (ETH Transfers to AssetHub; CCRMS Not Exercised)

Step	Component	Action
1	Geth	Transaction <code>Gateway.sendToken(1 ETH, Alice)</code> included in block 5157
2	Lodestar	Block finalized at epoch ≥ 162
3	Beacon Relay	Generated inclusion proof, submitted beacon header update
4	Bridge Hub	<code>ethereumBeaconClient.submit(...)</code> verified BLS signatures, advanced <code>latestFinalizedBlockRoot</code>
5	Execution Relay	Generated receipt + execution + ancestry proof, submitted message bundle
6	Bridge Hub	<code>ethereumInboundQueue.submit(...)</code> verified the proof against the verified beacon state root
7	Bridge Hub → AssetHub	XCM message dispatched via HRMP
8	AssetHub	<code>foreignAssets.mint(Ether, Alice, 1 ETH)</code> succeeded

CCRMS was not on the path; the result confirms the local pipeline and the CCRMS runtime's location-converter configuration. The final hop to a CCRMS operation was not tested end-to-end (Section 8.5); integration challenges are in Appendix A.

7.1.8 | Comparative Analysis vs. Centralized Baseline

We benchmarked CCRMS against Express.js with SQL.js for the same six operations, using the same methodology. Table 7.7 presents the comparison.

Table 7.7. Comparative Analysis: CCRMS Parachain vs. Centralized (Express.js + SQLite)

Metric	CCRMS Parachain	Centralized	Ratio
Sustained TPS	26.2	7,305	270×
Operation latency (server-side)	~6,000 ms	0.09 ms	67,000×
Stress test success rate	22% (RPC nonce conflicts)	100%	–
Storage per content	191 bytes	~100–150 bytes	1.5×
MTTR	15–20 seconds	2–3 seconds	7×
Cross-chain equivalent	~100 seconds	5–50 ms (HTTP request)	2,000–20,000×

The 22% stress-test figure in Table 7.7 is the client-visible success rate over the full saturation window, including the nonce-exhaustion burst described in Section 7.1.2. It is not the 96.9% happy-path inclusion rate used in the abstract and Section 7.1.2, nor is it a consensus failure rate.

The centralized implementation outperforms CCRMS across all quantitative metrics (270× TPS, 67,000× latency). CCRMS’s qualitative properties (no single point of failure, cross-chain support, provable rights state, and censorship resistance) are not reflected in quantitative data but are central to the use case (gaps G1 and G7, Chapter 2). At 26.2 TPS (≈ 2.3 million operations per day), CCRMS suffices for creator platforms where rights transactions are low-frequency events.

7.2 | Economic Results

The economic results address SQ5 using Monte Carlo simulation, the audience-size crossover finding, and a comparative fee analysis.

7.2.1 | Monte Carlo Creator Revenue Simulation

The simulation models 1,000 creators across 10,000 iterations under four platform configurations, projecting one-year revenue using platform-specific fees, algorithmic discovery, and monthly churn. It uses a seeded pseudo-random number generator (xoshiro128**,

seed 42), so every figure is exactly reproducible. Full parameters are in Appendix G; sensitivity analysis in Section 7.2.4.

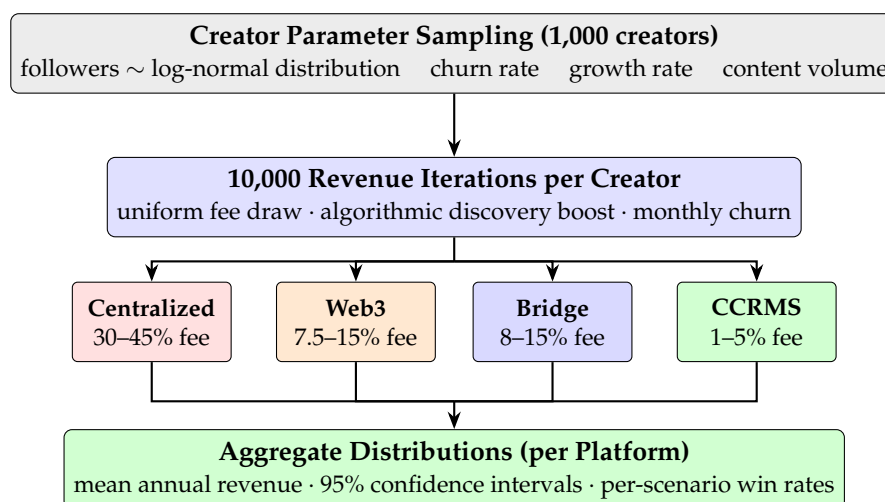


Figure 7.1. Monte Carlo simulation structure: 1,000 creators × 10,000 iterations = 10 million revenue paths per platform. Platform fee drawn uniformly each iteration; full parameters in Appendix G.

Table 7.8. Monte Carlo Creator Revenue Simulation (Mean Annual Revenue Per Creator)

Platform Model	Fee Range	Mean Revenue	vs. CCRMS
Centralized (YouTube/Spotify-style)	30–45%	\$45,416	CCRMS +16.8%
Web3 marketplace (OpenSea-style)	7.5–15%	\$48,711	CCRMS +8.9%
Bridge-based cross-chain	8–15%	\$45,369	CCRMS +16.9%
CCRMS	1–5%	\$53,053	Baseline

7.2.2 | Audience Size Dependency

CCRMS’s mean advantage (Table 7.8) is not uniform: it is heavily dependent on **audience size**. Table 7.9 breaks results down by tier.

Table 7.9. CCRMS Win Rate Against Centralized Platform by Audience Tier

Tier	Followers	Win rate vs centralized	Notes
Tiny	<100	0.0%	Centralized algorithmic discovery dominates
Small	100–500	0.0%	Fee savings insufficient to offset discovery
Medium	500–2,000	6.6%	Transition begins
Large	2,000–10,000	57.6%	Crossover; the 50% point falls near 3,600 followers
Huge	10,000–100,000	99.0%	CCRMS strongly preferred
Mega	100,000+	99.9%	CCRMS near-certain advantage

For creators with fewer than ~3,600 followers (transition beginning near 2,000 where the win rate reaches 6.6%), centralized discovery outweighs fee savings. For those with 10,000 or more followers, the fee differential dominates and CCRMS is preferred in over 99% of scenarios.

These results describe the CCRMS design; the prototype’s fifty-holder bound (Section 8.5) must be lifted before the mid-size and large tiers can be served.

Table 7.10 reports the per-scenario break-even rates.

Table 7.10. Per-Scenario Break-Even Rates

Comparison	CCRMS Per-Scenario Win Rate
CCRMS vs Centralized	28.0%
CCRMS vs Web3 marketplace	38.6%
CCRMS vs Bridge-based cross-chain	66.5%

Per-scenario win rates are lower because CCRMS loses in small-audience scenarios; the 66.5% win rate against bridge-based cross-chain reflects shared fee assumptions.

Table 7.11 reports 95% confidence intervals for total simulated revenue.

Table 7.11. 95% Confidence Intervals for Total Simulated Revenue (1,000 Creators)

Platform	Total Revenue Range
Centralized	\$30.9M–\$69.1M
Web3 marketplace	\$35.1M–\$78.4M
Bridge-based cross-chain	\$30.7M–\$69.4M
CCRMS	\$36.7M–\$84.1M

These intervals overlap across all platforms; the per-scenario win rates in Table 7.9 provide the stronger comparison.

7.2.3 | Comparative Fee Analysis

Table 7.12. Published fee ranges by platform category. Centralized literature values span 30–60%; the simulation uses 30–45% (Appendix G, Table G.2).

Category	Fee Range	Representative Platforms
Centralized	30–60%	YouTube (Google LLC, 2025), Spotify (Spotify AB, 2025), Apple Music (Apple Inc., 2025), Audible (Amazon.com, Inc., 2025)
Web3 marketplace	7.5–15%	OpenSea (2.5% protocol fee (OpenSea, 2025)), Magic Eden (2% (Magic Eden, 2025))
Bridge-based cross-chain	8–15%	Wormhole, LayerZero, Multichain (bridge + DEX swap fees; varies by route)
CCRMS	1–5%	Polkadot transaction fees (target), conservative range

7.2.4 | Sensitivity Analysis

A sensitivity analysis (Appendix G) finds the advantage insensitive to pricing and conversion parameters, moderately sensitive to subscription period and adoption rate, and strongly sensitive to audience distribution (a median of 500 followers eliminates it). The advantage persists for any CCRMS fee below approximately 17%.

7.2.5 | Prototype-Bound versus Simulated Cohorts

The win-rate results in Tables 7.9 and 7.10 characterize the CCRMS design. They do not characterize the artifact as benchmarked.

The prototype stores each first-time buyer as a child NFT within a `BoundedVec<_, ConstU32<50>>`. Expired subscriptions do not free their slots. Consequently, a single content item can accumulate at most fifty subscribers and owners over its lifetime (Appendix A.8). The simulation's Large, Huge, and Mega tiers assume no such cap. These are the tiers in which CCRMS wins (Table 7.9). Until the mitigation in Appendix E.4 is implemented, the prototype cannot host the audiences for which the fee-side advantage is claimed.

What the current artifact supports is independent of that cap: the three rights maps and the priority `check_access` (Claim 1, Section 1.5.1); the fee assumptions used in the simulation; and the issuance-style XCM on local HRMP (Claim 2). The economic figures remain a design-level result under Appendix G (Claim 3, Section 1.5.1).

7.3 | Security and User-Centric Results

7.3.1 | Security Audit Findings

The security audit (Scenario 6, Section 6.4) produced nine findings labeled A through I. Table 7.13 summarizes them.

Table 7.13. Security Audit Findings

ID	Severity	Description	Status
A	Low	XCM extrinsics callable locally as gift transactions (intentional)	Accepted design
B	High	Authorization gap in <code>xcm_transfer_ownership</code> : any funded account could transfer any user's ownership token	Fixed (commit 9ceca57)

Continued on next page

Table 7.13 continued from previous page

ID	Severity	Description	Status
C	Low	Permissive XCM filter configuration (SafeCallFilter, XcmExecuteFilter, XcmTeleportFilter, XcmReserveTransferFilter all Everything)	Accepted prototype config
D	Low	Zero-price content registration allowed	Accepted feature
E	Low	View pack overwrite on re-purchase	Fixed (commit 9ceca57)
F	Informational	Single-collator MTTR ~15–20 seconds	Test environment artifact
G	Positive	100% state persistence across crash	Confirmed
H	Positive	Block production rate unchanged after recovery	Confirmed
I	Medium (design limit)	Zero-view denial of service: purchase_views(num_views=0) charges \$0 but mints a child NFT, permanently exhausting a child slot	Open: unmitigated design-limit DoS; fix specified in Appendix E.4

Two findings (B and E) have been remediated; one (I) is an open Medium-severity design-limit denial-of-service that consumes a child slot at zero price (fix specified in Appendix E.4); three (A, C, D) are accepted design properties; and three (F, G, H) are positive reliability observations. **Finding B** and **Finding E** were addressed in commit 9ceca57; the Finding B remediation is conservative (the equality check cannot be satisfied by a remote XCM caller, so cross-chain transfers via `xcm_transfer_ownership` are pending a follow-on mitigation). Appendix D provides the complete narrative.

7.3.2 | Test Coverage

Our unit test suite covers **16 of the 19 distinct error variants** defined in `pallet-content-rights`, achieving 84 percent coverage, which exceeds the Phase 2 objective of 80 percent. Three variants remain untested and are out of scope for the unit test suite: `ItemIdOverflow` requires exhausting a u32 counter under normal extrinsic dispatch, which is not feasible

in a test environment; `NftOperationFailed` is triggered only by internal pallet-nfts state corruption that the mock runtime does not replicate; and `ChildAlreadyNested` cannot be reached through any publicly exposed extrinsic. The comprehensive error variant table is provided in Appendix D.

7.3.3 | Decentralization Assessment (HHI)

The Herfindahl-Hirschman Index (HHI) was calculated from the Polkadot mainnet validator distribution (parachains inherit relay-chain security). Comparison values for Ethereum PoS and Solana are our estimates from publicly reported stake distributions (Rated.network; Solana Compass); the U.S. DOJ Horizontal Merger Guidelines define HHI below 1,500 as unconcentrated (U.S. Department of Justice and Federal Trade Commission, 2010). Table 7.14 presents the comparison.

Table 7.14. Herfindahl-Hirschman Index (HHI) Comparison

System	Validators / Entities	HHI (approx.)	Nakamoto Coefficient
Polkadot mainnet (NPoS, ~600 validators, as of September 2026)	~600 (near-equal stake)	~16.7	~160–200
Ethereum PoS	~900K validators (concentrated in ~5 staking providers)	~1,200– 2,000	~5–7
Solana	~1,900 validators (top-heavy stake distribution)	~200–500	~19–25
Centralized DRM (e.g., Spotify)	1 entity	10,000	1
US DOJ “unconcentrated” threshold	–	<1,500	–

Polkadot’s HHI of ~16.7 (as of September 2026, ~600 validators) is well below the U.S. DOJ “unconcentrated” threshold of 1,500 and orders of magnitude lower than a centralized monopoly at 10,000. The Nakamoto coefficient (~160–200 for Polkadot vs.

~5–7 for Ethereum PoS and 1 for centralized DRM) confirms that CCRMS on Polkadot exceeds the decentralization of all alternatives by approximately 600-fold.

Table 7.14 should be read as a statement about safety under a production parachain, not about the test network. The relay-chain HHI indicates who secures CCRMS state once a block is included. The evaluated prototype’s liveness is limited to a single collator (Nakamoto coefficient 1 for block production). A deployed CCRMS would inherit Polkadot’s validator set for safety and would still need a collator set (or equivalent coretime) for liveness. The 600-fold HHI contrast with centralized DRM is therefore a property of the intended hosting environment, not of the Zombienet used in Chapter 7.

7.3.4 | Static Analysis and Dependency Audit

cargo clippy reported zero warnings. cargo audit reported 8 advisories in transitive SDK dependencies; none affect pallet code and all are mitigated by the sandboxed WASM runtime architecture (full list in Appendix D).

7.4 | Summary

Table 7.15 consolidates the measured outcomes for all 11 KPIs.

Table 7.15. KPI Results Summary

#	KPI	Target	Measured	Target met?
1	Throughput	>10 TPS	26.2 TPS sustained	Yes
2	Inclusion latency	<12 s	~6 s (1 block)	Yes
3	XCM inclusion delay	<10 blocks	0–7 blocks (median 5)	Yes
4	Block utilisation	<75%	12.9% at 300 txs (reference time only)	Yes — utilisation headroom confirmed
5	Storage efficiency	<500 bytes	191 bytes/content, 112 bytes/subscriber	Yes
6	Uptime	>85%	90%	Yes

Continued on next page

Table 7.15 continued from previous page

#	KPI	Target	Measured	Target met?
7	MTTR	<60 s	~15–20 s	Yes
8	State persistence	100%	100%	Yes
9	Security findings	No Critical/High unremediated at submission	1 High (Finding B) identified and patched by the author; 0 unremediated Critical/High findings in that audit. No third-party review (Section 1.5.3). Finding I remains an unmitigated design-limit DoS (Table 7.13, Appendix E.4).	Met as interpreted: no unremediated Critical/High findings in the author’s audit. Not independently verified.
10	Test coverage	>80%	84% (16/19 error variants)	Yes
11	Decentralization ratio	Documented	270× slower, qualitatively superior	Yes

Ten of eleven KPIs met their Table 6.4 targets for the stated measurements. KPI 9 is recorded as met only under the interpretation “no unremediated Critical/High in the author’s audit”; there was no third-party review, and Finding I remains open as a design-limit DoS (Section 8.5, Appendix E.4).

Discussion

This chapter analyzes the findings of Chapter 7 by identifying the questions addressed, the decisions substantiated, and the limitations that shape the interpretation of the contribution.

8.1 | Answering the Research Questions

8.1.1 | SQ1: Cross-Chain Rights Operations and Finality

The key finding is that the payer–beneficiary decoupling pattern, combined with XCM’s `Transact` instruction, is sufficient to delegate rights issuance to the authoritative chain without requiring two-phase commit. The remote parachain pays; CCRMS issues the rights record; no distributed rollback is needed because rights state is always authoritative on one chain.

Affirmative with one caveat. XCM v3 is sufficient for executing cross-chain rights operations: the canonical sequence `WithdrawAsset` → `BuyExecution` → `Transact` covers all five `xcm_*` extrinsics and achieves operations that are atomic within each chain’s execution; XCM `Transact` does not guarantee atomicity across chains (Appendix D, Finding A). The destination-chain inclusion delay was measured 0 to 7 blocks (median 5) (Section 7.1). No custom XCM extensions are required. The only caveat is that cross-chain recurring renewals depend on the readiness of XCM v5 `Schedule` production; in the meantime, an on-chain block hook manages local auto-renewal. This demonstration is inter-parachain (local HRMP). It is not an end-to-end Ethereum → CCRMS rights path (Section 1.5.2).

8.1.2 | SQ2: Latency versus Decentralization Trade-Offs

The trade-off is real but highly asymmetric: CCRMS operational latency is approximately $67,000\times$ higher than the centralized baseline (Table 7.7: $\sim 6,000$ ms vs. 0.09 ms), while a production parachain would inherit an HHI of ~ 16.7 compared with 10,000 for a centralized monopoly (Table 7.14), a decentralization gain of roughly two orders of magnitude on that index, for safety, not for test-network liveness (Section 7.3.3). For discrete, low-frequency rights operations (the CCRMS use case), the trade-off is advantageous; for high-frequency interactive applications it is not.

8.1.3 | SQ3: Unified Rights Record Enforcing All Three Monetization Models

Yes. The unified primitive works (Section 5.1): three independent storage maps with a priority-resolution access check enforce all three access semantics without novel cryptographic mechanisms, closing gap G7. Story Protocol's PIL is a generic licensing primitive configured per policy; CCRMS is a specialized access-mode primitive (one Content record, three maps, one ordered check, and renewal/expiry as storage transitions). That is the claimed distinction; we do not claim PIL cannot be configured for similar commercial outcomes. Royalty propagation applies uniformly to all primary-market revenue. Cost savings are substantial, but audience-dependent (Section 7.2; Section 8.2.2).

8.1.4 | SQ4: Security and Authorization

`xcm_transfer_ownership` accepted a `from` identity parameter without verifying that the caller held it; a one-line `ensure!` check (Finding B, Section 7.3.1) remediated the issue. The lesson generalizes: **cross-chain extrinsics that accept identity parameters require explicit authorization checks.**

8.1.5 | SQ5: Economic Viability under Realistic Creator Audience Distributions

Yes, conditionally. The Monte Carlo simulation (Section 7.2) indicates, at the level of the design, that CCRMS yields higher mean creator revenue across all four simulated platform configurations, with an advantage ranging from 8.9% to 16.9%. The result is robust to the fee range (advantage persists for any CCRMS fee below 17%; Appendix G) but depends on the assumed audience distribution. This advantage depends heavily on the size of the creator's audience. We explore the crossover point and its implications in

Section 8.2.2. Our contribution is to transform this conditionality into a **quantitative and reproducible** framework rather than merely qualitative observations.

8.2 | Interpretation of Key Results

8.2.1 | The 26.2 TPS Figure in Context

The figure of 26.2 TPS reflects a single-collator local Zombienet configuration that does not exercise the throughput features of Polkadot 2.0 (Agile Coretime, Asynchronous Backing, and Elastic Scaling were available in the `stable2512` SDK but not enabled in the chain specification), and does not represent the architecture's maximum capacity. Polkadot achieves scalability through **horizontal** scaling: with 100 parachains at 26 TPS each, the total capacity reaches 2,600 TPS (Parity Technologies, 2021; Wood, 2016). The 26.2 TPS figure is consistent with the range reported for public blockchain systems at comparable scale (Rao et al., 2024), and horizontal scaling across multiple chains is the principal path to higher aggregate throughput. More importantly, the $270\times$ ratio relative to the centralized baseline is misleading. CCRMS is designed to process low-frequency events; it records discrete rights transactions with cryptographic finality, automatic royalty propagation, and cross-chain portability, none of which the centralized baseline provides at any throughput level. When assessed against its actual workload, 26.2 TPS comfortably exceeds the demands of every creator-economy platform.

8.2.2 | The Economic Crossover at $\sim 3,600$ Followers

The economic advantage of CCRMS primarily benefits creators with established audiences. For those with fewer than $\sim 3,600$ followers (with the transition beginning near 2,000 followers where the win rate reaches 6.6%), centralized platforms tend to generate greater revenue because algorithmic discovery helps creators reach audiences they might not otherwise reach. While this initially appears to challenge the notion that CCRMS serves as a creator-friendly alternative, the more accurate and credible assertion is conditional: (decentralized = better for creators **with established audiences**). CCRMS is primarily a tool designed **for creators who have surpassed the discovery benefits of centralized platforms and seek to maximize revenue retention**.

8.3 | Architectural Decisions Revisited

The pallet-first decision proved well-founded. The pivot was driven by three engineering constraints (Appendix A.2), and it was taken against the backdrop of the January 2026 announcement that ink! development would be discontinued, which removed any expectation that the contract layer would mature. The episode illustrates that architectural decisions reducing dependence on any single non-essential component become more valuable as the ecosystem evolves (Bass et al., 2021; Parnas, 1972). This principle is also reflected in the choice of pallet-nfts over the frozen RMRK pallets and in HRMP rather than custom inter-parachain protocols. The same pattern holds for the Snowbridge integration: the configuration required for Ethereum bridging was completed at minimal additional cost, though the Ethereum-to-CCRMS path itself remains untested.

The unified rights record and the payer-beneficiary pattern are likewise vindicated: the former exhibits linear storage growth (fifty-holder cap noted in Section 8.5), and the latter is sound subject to Finding B’s authorization requirement.

8.4 | The Work in Context

The Polkadot ecosystem evolved during implementation. **Polkadot 2.0** features reached production by late 2025, indicating that our latency metrics are pre-2.0 upper bounds; deployment uses the updated SDK with no pallet-level changes. **Snowbridge V2** is backward-compatible with the V1 prototype. **DATA Foundation** (formerly Story Protocol, rebranded June 2026 to focus on AI training-data provenance) strengthens the case that the cross-chain creator-rights niche remains unserved; comparisons with the Programmable IP License remain relevant. **EU regulation** (AI Act, MiCA) drives demand for cryptographic provenance; MiCA’s treatment of subscription tokens remains legally unclear. Cross-chain interoperability surveys (Deng et al., 2025; Li et al., 2025; Maric et al., 2025) provide context but do not change the conclusions.

8.5 | Limitations

The CCRMS prototype has six principal limitations:

1. **No production deployment.** All measurements are from a local Zombienet topology with two relay-chain validators, single-collator parachains, and no real-world latency. Chapter 7 results are upper bounds in this test environment and do not predict production behavior.

2. **Single-author validation.** A single author performed the implementation, test design, measurement collection, and interpretation. No independent code review, third-party security audit, or external benchmark validation has been conducted. Production deployment would require all three before any non-trivial value passes through the system.
3. **Local-only Snowbridge integration.** Geth and Lodestar ran on the same machine as the Polkadot side, avoiding many realities of public Ethereum (variable block times, MEV, network contention, beacon chain reorgs). The integration serves as a proof of architectural feasibility but provides no data on production bridge behavior.
4. **The fifty-holder bound.** The `BoundedVec<_, ConstU32<50>>` bound on `Children` limits each content item to fifty child NFTs. Because each first-time purchase mints one for the buyer and expired subscriptions never release theirs, a content item can serve up to fifty subscribers and owners over its lifetime. This places the audience tiers that benefit most (Section 7.2; Section 7.2.5) beyond the current instantiation. Finding I is the low-cost exploit of that bound: `purchase_views(num_views = 0)` mints a child at zero price (Table 7.13). The bound is not intrinsic to the design; Appendix E.4 describes the mitigation paths, none of which are yet implemented. The Monte Carlo result therefore characterizes the intended design, not the bounded prototype (Claim 3, Section 1.5.1).
5. **No empirical proof of the bridged-payment final hop.** The bridged-ETH-pays-for-CCRMS-operation flow is conceptually straightforward but has not been tested end-to-end. We claim forward-compatibility with the flow, which is a weaker claim than an empirical demonstration would provide.
6. **Simulation input sensitivity.** Simulated outcomes are robust to pricing and fee parameters but depend strongly on the assumed audience distribution and value of algorithmic discovery, neither validated against real data; consumer onboarding friction is unmodeled.

8.6 | Implications

The findings extend beyond the immediate prototype in four directions.

For platform designers, the overarching lesson is that **architectural choices that minimize reliance on any single non-essential component pay dividends during**

ecosystem-level changes. Two corollaries: **cross-chain extrinsics that accept identity parameters require explicit authorization audits** (Finding B), and **reimplementing an unmaintained standard is preferable to depending on it** (the RMRK 2.0 pivot).

For creators, the Monte Carlo simulation suggests that the question “Is a decentralized platform actually better for me?” has a conditional answer: **yes, depending on audience size.** Creators with over 10,000 followers substantially benefit from lower fees; those with fewer than ~2,000 are better served by centralized discovery. The choice should reflect audience size.

For regulators, on-chain rights management systems should be treated as **compliance infrastructure** for emerging copyright and AI training regimes rather than as alternative payment platforms. The open question about MiCA’s classification of subscription tokens with fungible characteristics would benefit from regulatory clarity, as the current ambiguity poses risks for would-be EU deployers.

For the research community, the prototype, implementation specifications, and Monte Carlo simulation are intended to support follow-up research. The methodological contribution is a worked DSR example for multi-component blockchain prototyping. The empirical contribution is the audience-dependent finding, which future research could refine using real-world migration data or models that incorporate creator psychology and switching costs.

8.7 | Critical Self-Reflection

In retrospect, we would have **initiated the Snowbridge integration earlier**; the setup complexity (Appendix A) required several days of unexpected debugging. We would also have **prioritized FRAME benchmarking earlier** to report fee-adjusted throughput. We would **not** change the pallet-first decision, the unified rights record model, or the cross-chain operation pattern.

8.8 | Summary

The five research sub-questions are answered affirmatively; the most economically significant finding is the audience-dependent CCRMS advantage. The major architectural decisions are vindicated, most emphatically the pallet-first decision. The six principal limitations constrain how the contribution should be read but do not invalidate it. Chapter 9 presents the conclusion and future work.

Conclusion and Future Work

This closing chapter restates the contributions, summarizes the key findings, identifies the most important threads for future work, and offers a brief closing reflection.

9.1 | Summary of Contributions

Section 1.4 stated four contributions. We expand each here.

1. **A feasibility result for the unified rights primitive.** The central contribution shows that subscription, pay-per-view, and ownership can be encoded as native access modes of a single on-chain token — not as configurations of a licensing primitive, but as state transitions enforced by a unified access check. It uses three storage maps (Subscriptions, ViewPacks, Owners) with a single check and shared royalty logic; all tests and KPIs confirm correct enforcement. No cryptographic or consensus mechanisms are needed. This closes gap G7 and is the main academic contribution.
2. **Quantified decentralization–latency and revenue trade-offs.** Prior work endorsed decentralization qualitatively or measured only centralized performance. This dissertation quantifies the trade-offs directly. The latency overhead is $67,000\times$ and the throughput penalty is $270\times$ against a centralized Express.js baseline; the Herfindahl-Hirschman Index improvement is approximately 600-fold (Polkadot HHI ~ 16.7 , based on approximately 600 active validators as of September 2026, vs. centralized DRM at 10,000). The Monte Carlo simulation shows CCRMS yields a 16.8% higher mean revenue than the centralized baseline overall (Table 7.8), with the advantage concentrated among creators with 10,000 or more followers. Both are novel in blockchain content rights systems.

3. **A cross-chain rights artifact demonstrating XCM sufficiency.** The CCRMS parachain — pallet-content-rights, pallet-rights-verifier, an ink! 6 wrapper contract, an EVM read-only precompile, and benchmark and simulation scripts — demonstrates that XCM v3 with payer–beneficiary decoupling is sufficient for all five authenticated cross-chain rights operations without custom protocol extensions. Seven implementation specifications (pallet design, XCM message flows, security threat model, royalty algorithm, precompile interface, NFT nesting model, and Snowbridge integration — the latter covering the integration and its debugging findings, not end-to-end Ethereum-to-CCRMS compatibility, which remains untested) provide design knowledge intended to support future replication and extension. The artifact and specifications are released open source under the MIT license.
4. **A DSR worked example for multi-component blockchain prototypes.** The full DSR cycle — problem identification, design, instantiation, evaluation, and communication — is applied to a system with FRAME pallets, ink! contracts, XCM, Snowbridge, and Monte Carlo simulation. This combination is rarely seen in existing DSR literature, which typically focuses on single-component blockchain systems. Future researchers building similar multi-layer blockchain prototypes can use this dissertation as a methodological reference.

9.2 | Principal Findings

Six principal findings emerge from the work:

1. **The primary research question is answered in the affirmative:** a Polkadot parachain with XCM v3 and FRAME can deliver the unified rights record. Issuance and access work; Findings B and I still constrain ownership transfer and the child cap. Ten of eleven KPIs in Table 7.15 are met; KPI 9 is an author-audit interpretation only (Section 7.4).
2. **The major architectural decisions aged well.** The pallet-first decision was well-founded in light of the January 2026 ink! discontinuation; the RMRK 2.0 to pallet-nfts pivot, the unified rights record model, and the cross-chain operation pattern are also vindicated. In retrospect, we would not change these decisions.
3. **The economic advantage depends on audience size.** CCRMS benefits creators with 10,000+ followers but is less competitive than centralized platforms for those

with under ~3,600 followers due to algorithmic discovery (the transition begins near 2,000 followers). While CCRMS earns more on average, the conditional framing better reflects individual creator decisions. Because this advantage lies in audience sizes the current prototype cannot yet serve (Section 8.5), lifting the holder bound is a prerequisite for it to apply in practice. Therefore the Monte Carlo result characterizes the intended design; it is not an empirical property of the bounded prototype.

4. **Cross-chain operations are operationally feasible, but latency-bound.** XCM v3 is sufficient for issuance-style rights operations on local HRMP. The destination-chain inclusion delay was measured in a single-collator configuration that did not enable Async Backing; it is an upper bound for that environment, not a fundamental limit. Ethereum interoperability is a configured runtime path plus a local ETH → AssetHub transfer; the rights pallet was not on that path.
5. **Standards reimplementations are sometimes preferable to standards adoption** when the reference implementation is unmaintained; the RMRK 2.0 pivot exemplifies this.
6. **Forward compatibility is a defensive engineering goal.** Configuring the runtime to accept Ethereum-originated XCM messages at minimal additional cost in Phase 4 completed the Snowbridge integration configuration; the gindex and forkVersions debugging findings (Appendix A) are a useful engineering result, though the Ethereum-to-CCRMS path itself remains untested.

9.3 | Future Work

11 threads of future work emerged from the research. Appendix E provides full descriptions. The cross-chain secondary market is out of scope in the current prototype; Appendix E.8 describes the capability-based delegation approach for restoring `xcm_transfer_ownership` over XCM.

Table 9.1. Future Work Summary

#	Thread	Priority	Key action
E.1	Production deployment	Near-term	Deploy to public testnet/mainnet; obtain third-party security audit

Continued on next page

Table 9.1 continued from previous page

#	Thread	Priority	Key action
E.2	Polkadot 2.0 migration	Near-term	Target newer SDK release (no pallet changes); compress latency via Async Backing, Agile Coretime, Elastic Scaling
E.3	Smart contract layer migration	Near-term	Choose among Solidity rewrite, wrevive adoption, or contract removal (ink! discontinued Jan 2026)
E.4	50-child cap mitigation	Near-term	Replace the bounded Children vector with a counter or remove it; prerequisite for the economic findings to apply at scale
E.5	User study	Near-term	Build minimal wallet/CLI; test with Web3 creators and consumers (highest-priority research extension)
E.6	Bridged-payment final hop	Near-term	Configure PaymentCurrency for foreign Ether or add swap step
E.7	Resale royalties	Medium-term	Add optional <code>resale_royalty_basis_points</code> ; requires on-chain marketplace for price verification
E.8	Capability-based delegation	Medium-term	Relax Finding B remediation with time-limited delegate authorization
E.9	Monte Carlo extensions	Medium-term	Add creator psychology, network effects, longitudinal and regulatory modeling
E.10	Formal verification	Long-term	Apply Isabelle/HOL techniques to authorization model, royalty algorithm, cross-chain pattern
E.11	JAM transition	Long-term	Migrate to JAM (no mainnet date announced as of mid-2026; multi-client implementations completing early milestones); no rebuild required

9.4 | Closing Reflection

CCRMS started with the idea that creators deserve more value, and blockchain offers a way to achieve this. We built a prototype, tested its operations, and simulated its economic effects. The premise held but was refined: Monte Carlo simulations show that decentralization is better for creators who have outgrown centralized platforms' discovery value, not universally. An honest, conditional claim is more valuable than a universal one.

The Polkadot ecosystem evolved significantly during implementation, with major updates like Polkadot 2.0, Snowbridge V2, and others. None invalidated our work because we chose conservatively — favoring stable primitives, minimizing reliance on non-essential parts, and preferring proven deployment patterns. This highlights a study in **defensive engineering** and **conservative, well-documented architecture** as the most durable approach.

CCRMS is a prototype in a single blockchain ecosystem, addressing a specific industry's structural issues. It isn't the future of content rights management, nor do we claim it to be. However, it shows that content rights management can be built on open, decentralized, cryptographically verifiable infrastructure, serving as a starting point for others to build upon.

Implementation Challenge Narratives

The nine challenges summarized in Table 5.1 are presented here in full detail with Challenge, Response, and Lesson for each.

A.1 | The Pivot from RMRK 2.0 to `pallet-nfts`

Challenge. The original Phase 3 design envisioned using the RMRK 2.0 nested-NFT specification, implemented atop the `rmrk-substrate` pallet suite. RMRK 2.0 precisely offers the parent-child nesting semantics that the CCRMS unified rights record model requires. Reusing an established standard is intended to reduce implementation risk. However, during initial investigations in Phase 4, we discovered that the `rmrk-substrate` pallets were frozen on Polkadot SDK version 0.9.36 and relied on the now-deprecated `pallet-uniques` instead of the modern `pallet-nfts`. Our efforts to compile `rmrk-substrate` against the current Polkadot SDK monorepo were unsuccessful because of incompatible storage schemas and removed traits.

Response. We decided to replicate RMRK 2.0's nesting semantics directly on top of `pallet-nfts` without depending on any RMRK pallets. This involved designing a parent-child storage layout (the dual `Children` and `Parent` index), implementing the lifecycle helpers (`mint_and_nest_child`, the burn and re-mint patterns in `transfer_ownership`), and tagging child NFTs with a `rights_type` attribute via `pallet_nfts::set_attribute`. The implementation took longer than reusing `rmrk-substrate` would have, but the result is fully integrated with the modern Polkadot SDK and depends only on actively maintained pallets.

Lesson. Established libraries pose latent compatibility risks when they are not kept up to date with the current ecosystem. Reimplementing essential primitives from scratch

was justified by the maintainability gain and by the opportunity to demonstrate that RMRK-style composability does not require RMRK itself.

A.2 | The Pivot from Contract-First to Pallet-First Architecture

Challenge. The Phase 3 design designated ink! smart contracts as the primary interface for users, featuring three contracts (each corresponding to a different monetization model) to manage user operations. A chain extension mechanism was incorporated to facilitate access to a lightweight pallet dedicated to state coordination. During Phase 4 implementation, two issues arose concurrently: first, the chain extension mechanism within the current pallet-revive release was effectively non-operational for write activities; second, the operational complexity of debugging cross-pallet contract calls was significantly greater than that of direct FRAME pallet interactions.

Response. We inverted the architecture. The pallet (pallet-content-rights) became the definitive source of truth for all business logic, whereas the ink! contract (rights_manager) was retained as a streamlined user-facing interface that mirrors the pallet's API but holds no independent storage. We developed a custom EVM precompile (ContentRightsPrecompile) to provide read-only access to pallet state for Solidity-based clients, replacing the original chain extension design.

Lesson. The pallet-first decision proved well-founded. The pivot was driven by engineering constraints, and it was taken against the backdrop of the January 2026 announcement that ink! development would be discontinued, which removed any expectation that the contract layer would mature. The episode illustrates that architectural decisions reducing dependence on any single non-essential component become more valuable as the ecosystem evolves.

A.3 | The Snowbridge Fork-Version Configuration Issue

Challenge. During the integration of Snowbridge, the beacon relay generated Merkle inclusion proofs at incorrect tree positions (gindex 54 instead of 86 for the Electra fork), leading to the rejection of all submitted updates by the Bridge Hub's EthereumBeaconClient pallet.

Response. We identified the root cause as a misinterpretation of the relay's forkVersions configuration field. The relay code interprets this field as a fork activation

epoch rather than a version identifier. Consequently, `0x04000000` was interpreted as the integer 83,886,080, placing the active fork many epochs in the future. We resolved the issue by updating the configuration to use plain epoch numbers (`{ "deneb": 0, "electra": 0, "fulu": 5000000 }`).

Lesson. Cryptographic verification systems are sensitive to silent configuration discrepancies: the failure mode is “verification fails” with no informative error pointing to the bad value.

A.4 | The Lodestar Preset Version Pinning

Challenge. The `EthereumBeaconClient` pallet within Bridge Hub is compiled with `pubkeys: [PublicKey; 512]`. Any beacon header update originating from a smaller sync committee (e.g., the 32-validator minimal preset used by Lodestar’s development mode) is rejected during deserialization. Configuring Lodestar v1.41.0 with `LODESTAR_PRESET=mainnet` proved ineffective; the development mode silently disregarded the environment variable.

Response. Lodestar v1.35.0, when built from source, adheres to the `LODESTAR_PRESET=mainnet` variable. We modified the setup scripts to clone, build, and pin v1.35.0.

Lesson. Cryptographic light clients impose strict requirements that cannot be circumvented through configuration alone. Toolchain compatibility for a research-stage bridge is fragile and must be reconfirmed with each component upgrade.

A.5 | The XCM Sovereign Account Endianness Bug

Challenge. Our initial cross-chain XCM tests revealed a perplexing failure pattern: the source parachain reported successful XCM dispatch and balance deduction, yet the target parachain indicated `InsufficientPayment`.

Response. The source chain constructed the sovereign account using `polkadot.js`’s `toHex()` in default big-endian mode, whereas the CCRMS pallet expected little-endian byte order (the `SiblingParachainConvertsVia` convention). This produced two separate accounts: one funded but unused, one used but unfunded. We resolved the issue by changing to `toHex(true)`.

Lesson. Endianness bugs are particularly insidious because both orderings produce valid 32-byte addresses and the symptom (insufficient balance) misdirects the investigation.

A.6 | The XCM Fee Scaling for Proof Size

Challenge. Initial XCM tests configured the `BuyExecution` fee at 1B tokens. All tests failed; only at ~70B tokens did they begin to pass.

Response. The dominant cost in the runtime's `BlockRatioFee` model is `proof_size`, not `ref_time`. The standard configuration imposes significantly steeper scaling for proof size, meaning even a minimal cross-chain message requires 75–100B tokens. We updated test scripts to specify 100B tokens.

Lesson. Polkadot fee modeling is driven primarily by proof size, not computational effort.

A.7 | Finding B: The Authorization Gap in

`xcm_transfer_ownership`

Challenge. During the Phase 5 security audit, we identified an authorization flaw in `xcm_transfer_ownership`: the initial implementation performed only `ensure_signed(origin)` before burning/minting NFTs based on a `from` parameter, allowing any signed account to steal ownership.

Response. We added `ensure!(authorizer == from, Error::::Unauthorized)` and committed a unit test (`security_xcm_transfer_ownership_any_account_can_steal`) that verifies the exploit is blocked.

Lesson. Any cross-chain extrinsic accepting a “from” identity parameter requires explicit authorization. Payer-beneficiary separation introduces impersonation risk absent from local-only calls.

A.8 | The Auto-Renewal `MaxChildrenReached` Boundary

Challenge. Batches of 100 `subscribe` calls to the same content item failed midway with `MaxChildrenReached`. The `Children` storage entry is limited to 50 child NFTs by `BoundedVec<_, ConstU32<50>>`.

Response. We documented the constraint as a known limitation and updated benchmark scripts to rotate content items. Subsequent analysis showed the bound to be stricter than a concurrency limit. Children are minted per buyer and released only when a view pack is exhausted, so expired subscriptions permanently occupy their slots. The

bound therefore limits a content item to fifty subscribers and owners over its lifetime. Its implications for the economic results are discussed in Section 8.5.

Lesson. Bounded FRAME collections trade benchmarkability for capacity limits, and a bound that suits a benchmark may conflict with the target workload. Bounds should be checked against the audience sizes the system is meant to serve, and redundant indexes should be removed rather than bounded.

A.9 | The ink! Discontinuation and Its Implications

Challenge. On 27 January 2026, the ink! Alliance announced the cessation of ink! development following three failed OpenGov treasury funding proposals. Both the Web3 Foundation and Parity Technologies declined continued funding. The announcement outlined three migration options: Solidity via revive, low-level Rust against pallet-revive, or the community-maintained wrevive SDK.

Response. Because our pallet-first architecture (Appendix A.2) centers business logic in a FRAME pallet rather than in ink! contracts, the discontinuation has minimal practical impact. The ink! code in `contracts/rights_manager` serves as a non-load-bearing wrapper. We deferred migration.

Lesson. Architectural decisions that minimize dependence on any single non-essential component provide advantages during ecosystem-wide changes.

Research Objective Details

B.1 | RO1: Design and Prototype XCM Extensions

Statement. Design and prototype the use of XCM messages to transmit recurring rights operations and comprehensive rights metadata across Polkadot parachains. Evaluate atomic success rates and cross-chain finality durations.

Sub-questions: SQ1 primarily; SQ2 indirectly.

Deliverable: Functioning prototype with documented XCM latency and success rate metrics.

B.2 | RO2: Implement a Unified Rights Token

Statement. Implement a unified rights record (originally ink!, revised to FRAME pallet in Phase 4) with off-chain indexing support and access-verification latency measurement.

Sub-questions: SQ2, SQ3.

Deliverable: pallet-content-rights enforcing all three monetization models with documented latency.

Note: Off-chain indexer (SubQuery) was not implemented; pallet events suffice for on-chain queries via polkadot-js API. Acknowledged as a limitation.

B.3 | RO3: Merge Subscription, PPV, and Purchase Logic

Statement. Develop and integrate subscription, pay-per-view, and ownership transfer mechanisms into a unified, composable rights framework.

Sub-questions: SQ3, SQ4.

Deliverable: Single pallet implementing all three models through a unified API, with documented trade-offs.

B.4 | RO4: Benchmark Scalability, Interoperability, and Economic Efficiency

Statement. Benchmark against centralized DRM, bridge-based solutions, and (originally) Ethereum L2s.

Sub-questions: SQ2, SQ5.

Deliverable: Comparative TPS, latency, and creator revenue retention analysis.

Note: Ethereum L2 target was substituted with Express.js + SQLite local benchmark in Phase 5. Core analysis is intact.

B.5 | RO5: Zero-Knowledge Proof Hooks (Not Achieved)

Statement. Integrate optional ZK proof hooks for selective regulatory disclosure.

Status: Not achieved. Integrating a ZK proof system into a Substrate runtime (selecting a proof system, building a verifier pallet, designing a disclosure protocol, producing test circuits) was substantially more complex than anticipated. Restated as future work in Section 9.3.

B.6 | RO6: Quantify Creator Savings via Monte Carlo Simulation

Statement. Quantify creator savings through Monte Carlo simulation comparing CCRMS with centralized (30–45% fees), Web3 marketplace, and bridge-based (8–15% fees) alternatives.

Sub-questions: SQ5.

Deliverable: Monte Carlo simulation with documented input distributions, model assumptions, and comparative output statistics.

Note: Phase 2 listed this as potentially replaceable by centralized benchmarking. Phase 5 included both; RO6 is fully achieved.

Code Listings

C.1 | Solidity Interface: ContentRightsPrecompile

The EVM precompile (Section 5.1) exposes the following Solidity interface. Solidity contracts deployed on pallet-revive can call these methods to read CCRMS rights state without crossing the EVM/Substrate boundary at write time.

```
1 interface IContentRights {
2     function checkAccess(uint32 contentId, address who)
3         external view returns (uint8 rightsType);
4
5     function getContent(uint32 contentId) external view returns (
6         address creator,
7         bytes32 metadataHash,
8         uint128 subscriptionPrice,
9         uint128 ppvPrice,
10        uint128 ownershipPrice,
11        uint32 periodLength
12    );
13
14    function isOwner(uint32 contentId, address who)
15        external view returns (bool owned);
16
17    function getSubscription(uint32 contentId, address subscriber)
18        external view returns (bool exists, uint32 expiryBlock);
19
20    function getViewPack(uint32 contentId, address viewer)
21        external view returns (bool exists, uint32 viewsRemaining);
22 }
```

C.2 | Canonical XCM Message for Cross-Chain Rights Operations

The canonical XCM message constructed by the `xcm_*` extrinsics (Section 5.2.1) follows the three-instruction `WithdrawAsset` $\rightarrow$ `BuyExecution` $\rightarrow$ `Transact` pattern. The JavaScript form below illustrates the structure as submitted via `@polkadot/api`.

```

1 const xcmMessage = {
2   V3: [
3     { WithdrawAsset: [{
4       id: { Concrete: { parents: 1, interior: 'Here' } } },
5       fun: { Fungible: 100_000_000_000 }
6     } ] },
7     { BuyExecution: {
8       fees: {
9         id: { Concrete: { parents: 1, interior: 'Here' } } },
10        fun: { Fungible: 100_000_000_000 }
11      },
12      weightLimit: 'Unlimited'
13    } },
14    { Transact: {
15      originKind: 'SovereignAccount',
16      requireWeightAtMost: { refTime: 1_000_000_000, proofSize: 100_000 },
17      call: { encoded: encodedCall }
18    } }
19  ]
20 };

```

C.3 | Repository Directory Layout

The CCRMS parachain repository follows the standard Polkadot SDK template structure:

```

content-rights-parachain/
|-- pallets/
|   |-- content-rights/           # Core business logic and tests
|   '-- rights-verifier/         # Cross-chain Merkle proof verification
|-- runtime/
|   '-- src/
|       |-- lib.rs               # Runtime composition
|       |-- configs/             # Pallet configurations and XCM
|       '-- precompiles.rs       # Custom EVM precompile
|-- contracts/

```

```
|  '-- rights_manager/      # ink! 6 contract
|-- node/                  # Collator binary
|-- scripts/               # Benchmarking, testing, deployment
'-- docs/                  # Setup and integration documentation
```

C.4 | Dispatchable Extrinsic Reference

Table C.1 lists all 17 dispatchable extrinsics of pallet-content-rights.

Table C.1. Dispatchable Extrinsic in pallet-content-rights

Index	Extrinsic	Purpose
0	register_content	Create a new content item, instantiate the underlying NFT collection, mint the parent Content NFT
1	subscribe	Purchase a new subscription
2	renew_subscription	Renew an expired subscription
3	purchase_views	Purchase a pay-per-view pack of N views
4	consume_view	Decrement a view from an existing pack
5	purchase_ownership	Purchase permanent ownership
6	check_access	Verify whether a user holds any active rights
7	xcm_subscribe	Cross-chain subscription with payer-beneficiary decoupling
8	xcm_renew_subscription	Cross-chain renewal
9	xcm_purchase_views	Cross-chain pay-per-view purchase
10	xcm_purchase_ownership	Cross-chain ownership purchase
11	transfer_ownership	Peer-to-peer secondary-market ownership transfer
12	xcm_transfer_ownership	Cross-chain ownership transfer (caller-must-be-owner)

Continued on next page

Table C.1 continued from previous page

Index	Extrinsic	Purpose
13	set_royalty_splits	Configure the royalty distribution table (creator-only)
14	enable_auto_renew	Enable automatic renewal for an existing subscription
15	disable_auto_renew	Disable automatic renewal
16	query_rights_metadata	Emit a self-describing metadata event for cross-chain consumption

C.5 | Pallet Storage Layout Reference

Table C.2 lists all 10 storage items of pallet-content-rights.

Table C.2. Storage Layout of pallet-content-rights

Storage	Key → Value	Purpose
NextContentId	→ u32	Global content ID counter
NextItemId	collection_id → u32	Per-collection child item counter
Contents	content_id → ContentMetadata	Master record per content
Subscriptions	(content_id, user) → SubscriptionInfo	Active subscription state
ViewPacks	(content_id, user) → ViewPackInfo	Pay-per-view balance
Ownership	(content_id, user) → OwnershipInfo	Permanent ownership records
Children	(collection_id, parent) → BoundedVec<50>	NFT nesting forward index
Parent	(collection_id, child) → (u32, u32)	NFT nesting reverse index

Continued on next page

Table C.2 continued from previous page

Storage	Key → Value	Purpose
AutoRenewIndex	(content_id, user) → bool	Auto-renew-enabled subscriptions
RoyaltySplits	content_id → BoundedVec<10>	Collaborator payout shares

Security and Coverage Findings

D.1 | Security Audit Finding B: Authorization Gap in `xcm_transfer_ownership`

The original implementation performed only `ensure_signed(origin)` and then looked up `Ownership[content_id, from]`, burned the seller's NFT, and minted a new one for `to`. No check confirmed that the dispatch caller had any relationship to the `from` account, so any signed account could call the extrinsic with arbitrary `from` and `to` arguments and transfer ownership from any user to itself.

We rate this finding **High**. Exploitation requires only a signed account with sufficient balance for transaction fees, no relationship to the victim, and no prior state; the impact is unauthorized transfer of a permanent, transferable ownership right. We initially recorded the finding as Medium on the grounds that the affected extrinsic is reachable only through the cross-chain path in normal operation, but Finding A establishes that the `xcm_*` extrinsics are also callable locally by any signed account, which removes that mitigation.

Our remediation was `ensure!(authorizer == from, Error::::Unauthorized)`, committed alongside `unit test security_xcm_transfer_ownership_any_account_can_steal`.

This remediation is deliberately conservative. Because an XCM Transact with `SovereignAccount` origin dispatches under the sending parachain's sovereign account, the equality check cannot be satisfied by a remote caller, so `xcm_transfer_ownership` no longer supports genuine cross-chain transfers and is equivalent to `transfer_ownership` invoked locally. We accepted the loss of functionality in preference to a delegated-authority model that we could not verify within the implementation window; Ap-

pendix E.8 describes the capability-based alternative.

D.2 | Security Audit Finding E: View Pack Overwrite

The original `purchase_views` always overwrote `ViewPacks[content_id, buyer]` with a new `ViewPackInfo`, so a user holding 10 unused views who purchased 5 more would end up with 5, not 15. Our remediation made purchases additive: if a view pack already exists, new views are added to the existing balance.

This remediation covers the local `purchase_views` path only. The XCM path (`xcm_purchase_views`) still overwrites `ViewPacks` unconditionally and mints a fresh child NFT on each call, orphaning the previous child. Extending the fix to the XCM path is deferred to future work.

D.3 | Accepted Design Findings (A, C, D)

Finding A: `xcm_*` extrinsics are callable locally as gift transactions (economically neutral; documented for transparency). **Finding C:** Runtime XCM `SafeCallFilter` is set to `Everything`; the `XcmExecuteFilter`, `XcmTeleportFilter`, and `XcmReserveTransferFilter` are similarly set to `Everything`, and `FixedWeightBounds` is used in place of measured XCM weights; production should restrict all four to explicit allowlists. **Finding D:** Content can be registered with all prices at zero (legitimate free-content feature, not a vulnerability).

D.4 | Finding I: Zero-View Denial of Service

Severity: Medium

Description. `purchase_views` accepts `num_views = 0`, charges `ppv_price × 0 = 0`, and still mints a child NFT. `consume_view` then fails with `NoViewsRemaining`, so the slot is never released. Fifty throwaway accounts can permanently block all future sales of any content item for the cost of transaction fees alone.

Impact. Denial of service: any user can exhaust the fifty-child cap on any content item, preventing all future purchases at negligible cost. This is the exploit path for the fifty-child bound discussed in Section 8.5.

Remediation. Add `ensure!(num_views > 0, Error::::InsufficientPayment)` at the start of `purchase_views` and `xcm_purchase_views`. A dedicated `ZeroViews` error variant would improve clarity.

Status: Open.

D.5 | Error Variant Coverage

Table D.1. Error Variant Coverage

Error Variant	Tested	Error Variant	Tested
ContentNotFound	Yes	MaxChildrenReached	Yes
NotContentCreator	Yes	ContentIdOverflow	Yes
SubscriptionAlreadyExists	Yes	ItemIdOverflow	No
SubscriptionNotFound	Yes	OwnershipNotFound	Yes
SubscriptionNotExpired	Yes	InvalidRoyaltySplits	Yes
ViewPackNotFound	Yes	AutoRenewAlreadyEnabled	Yes
NoViewsRemaining	Yes	AutoRenewNotEnabled	Yes
AlreadyOwned	Yes	Unauthorized	Yes
InsufficientPayment	Yes	NftOperationFailed	No
		ChildAlreadyNested	No
			(unreachable)

D.6 | Static Analysis Advisory Breakdown

Table D.2. Static Analysis Advisory Breakdown

Category	Package	Risk	Mitigation
Networking (2)	quinn-proto, ring	DoS, panic	Relay/p2p layer only; no runtime impact
WASM executor (4)	wasmtime	Resource exhaustion, panics, f64 segfault	Sandboxed WASM execution model
TLS (1)	rustls-webpki	CRL bypass	Node-side only
Logging (1)	tracing-subscriber	Log injection	Observer layer only

All 8 advisories are mitigated by the architectural separation between the WASM-sandboxed runtime and the node binary. None are in code we authored.

Future Work Details

E.1 | Production Deployment

The most significant future work is deployment to a public testnet or mainnet. This would yield realistic latency and throughput measurements under network conditions that include geographic distribution, real-world contention, and validator set diversity; multi-collator reliability data replacing the single-collator MTTR figure of 15–20 seconds; genuine cross-chain operation against other parachains; and the opportunity for third-party security audit that would address the single-author validation limitation.

E.2 | Migration to Polkadot 2.0 Features

Although Polkadot 2.0 features (Agile Coretime, Asynchronous Backing, Elastic Scaling) were available in the `stable2512` SDK (December 2025) used throughout the project, the local Zombienet configuration did not enable them; the throughput figures in Section 7.1.1 therefore reflect a single-collator setup rather than the capabilities of the full Polkadot 2.0 stack. Migration would not require any pallet-level changes (the runtime would target an updated chain specification) but would compress single-chain and cross-chain latency, allow the parachain to acquire block production capacity on demand, and increase throughput proportionally via multi-core scheduling. XCM v5 support for multi-hop messages is included in the same release.

E.3 | Migration Paths for the Smart Contract Layer

The January 2026 ink! discontinuation leaves `rights_manager` without upstream toolchain support. Three migration paths are available: (a) Solidity rewrite using the revive Solidity-to-PolkaVM compiler (broadest tooling support, requires Solidity reimplementations); (b) adoption of `wrevive`, the community-maintained ink! successor (preserves the Rust experience, introduces a community-project dependency); (c) removal of the contract layer entirely (zero maintenance cost, loses ink!-style API convenience). The right choice depends on the intended client base and available maintenance resources.

E.4 | The 50-Child Cap

The `Children` vector duplicates information already held in `Subscriptions`, `ViewPacks`, and `Ownership` (holders) and in `Parent` (nesting), so the simplest mitigation is to remove it or replace it with a counter.

Three mitigation paths: raise the bound and re-benchmark (trades worst-case extrinsic weight for capacity); replace the bounded vector with a counter plus a separate membership map (removes the cap but loses single-read enumeration); introduce hierarchical “rights bucket” NFTs (more complex query patterns but unbounded fan-out). Any production deployment expecting highly popular content items would need to address the cap as a priority.

Whichever path is chosen, subscription expiry should also release or burn the associated child NFT, so that capacity reflects current rather than historical holders.

E.5 | User Study and Consumer-Facing Application

A future iteration would build a minimal web wallet integration or CLI tool and conduct user testing with creators and consumers drawn from Web3 communities. Specific questions: Can a typical Web3 user successfully purchase a subscription, consume a view, and transfer ownership without prompting? Do users understand the three monetization models from the wallet UI alone? What error messages are most confusing? Are creators willing to migrate from existing platforms? This was originally planned for Phase 2 but was not executed because we built no front end; it is the highest-priority near-term research extension.

E.6 | The Bridged-Payment Final Hop

Closing the gap between bridged ETH on AssetHub and a paid CCRMS rights operation requires either configuring `PaymentCurrency` to accept the foreign Ether asset directly or inserting an explicit swap step to convert ETH to the native parachain token. Either approach is conceptually straightforward.

E.7 | Resale Royalties

The current `transfer_ownership` extrinsic has no creator share on secondary-market transfers. CCRMS could add an optional per-content `resale_royalty_basis_points` field; the implementation challenge is verifying the declared sale price, which suggests resale royalties would work best in conjunction with an on-chain marketplace pallet that records sale prices as part of the transfer transaction.

E.8 | Capability-Based Delegation for Cross-Chain Transfers

The conservative Finding B remediation rules out delegated transfers. A future iteration could relax this with a capability-based authorization model (the owner pre-authorizes a specific delegate for a specific operation and time window), which is well-explored territory in capability-based security. The implementation challenge is the storage layout for capability records and the revocation lifecycle.

E.9 | Monte Carlo Simulation Extensions

Future extensions could incorporate creator psychology (risk aversion, platform-switching costs, brand attachment), model network-effect dynamics at critical creator mass, add longitudinal modeling over multiple years, and model regulatory effects from the EU AI Act or MiCA enforcement.

E.10 | Formal Verification

Following Maric et al. (2025) on Isabelle/HOL verification of cross-chain bridges, similar techniques could be applied to the CCRMS authorization model, the royalty distribu-

tion algorithm, and the cross-chain operation pattern. Formal verification is expensive but offers guarantees that empirical testing cannot match, and is worthwhile for any production deployment handling significant value.

E.11 | JAM Transition

JAM (the Join-Accumulate Machine) represents Polkadot's architectural successor to the existing relay chain. The testnet was launched in January 2026 with the participation of multiple implementation teams; no mainnet date has been announced as of mid-2026, with multi-client implementations completing early milestones across several language runtimes. CCRMS does not incorporate any JAM-specific features, and our implementation specifications provide sufficiently detailed architectural decisions to facilitate potential future migration without necessitating a complete rebuild.

Work Plan and Timeline

We structured the work into six phases, originally planned across 24 weeks:

- **Phase 1 (Weeks 1–6): Research and Literature Review.** Establishes the academic and commercial context. Produces the initial 78-source literature review (later expanded to 100).
- **Phase 2 (Weeks 7–10): Design Conceptualization.** Translates high-level concepts into the system design and methodology.
- **Phase 3 (Weeks 11–12): Architecture Finalization.** Bridges the conceptual design to a concrete implementation plan.
- **Phase 4 (Weeks 13–16): Core Implementation.** The primary build phase: pallet-content-rights, runtime configuration, contract layer, XCM, Snowbridge integration.
- **Phase 5 (Weeks 17–20): Testing and Evaluation.** Unit tests, benchmarks, security audit, reliability test, comparative benchmark, Monte Carlo simulation, and literature review version 2.
- **Phase 6 (Weeks 21–24): Finalization.** Dissertation writing, implementation specifications, and LaTeX submission preparation.

References to “Phase N” throughout the dissertation denote work produced during that phase.

Monte Carlo Simulation Parameters and Sensitivity

This appendix documents the full parameter set and sensitivity results for the Monte Carlo revenue simulation described in Section 7.2. The simulation is seeded (xoshiro128**, seed 42) and exactly reproducible from the published repository.

The headline run used 1,000 creators $\times$ 10,000 iterations. The sensitivity and sweep results below used 200 creators $\times$ 2,000 iterations with the same seed; the baseline advantage reads 17.0% against the 16.8% in the headline results due to the smaller grid.

G.1 Stochastic Parameters

Table G.1 documents the per-creator stochastic parameters drawn each iteration.

Table G.1. Monte Carlo Stochastic Parameters (per creator, per iteration)

Parameter	Distribution
Base audience	Log-normal ($\mu = 1,000, \sigma = 2$)
Content price	Uniform (\$1–\$50)
Subscription adoption	Beta ($\alpha = 2, \beta = 5$), mean $\approx 28.6\%$
PPV conversion	Beta ($\alpha = 1, \beta = 10$), mean $\approx 9.1\%$
Secondary-sale probability	Exponential ($\lambda = 0.1$), capped at 1.0
Stream weights	Subscription 50%, PPV 30%, secondary 20%
Subscription period	12 months

Table G.2 documents the per-platform parameters drawn uniformly once per iteration.

Discovery is additive rather than multiplicative: a flat bonus of 500–3,000 followers matters substantially at 200 followers and marginally at 50,000, which is what produces the audience-size crossover.

Table G.2. Monte Carlo Platform Parameters (drawn uniformly once per iteration)

Platform	Fee	Discovery added	Monthly churn	Cost/tx
Centralized	30–45%	+500–3,000	2–5%	\$0.000
Web3 market-place	7.5–15%	+100–800	5–10%	\$0.005
Bridge-based	8–15%	+50–400	6–12%	\$0.020
CCRMS	1–5%	+0–200	4–8%	\$0.001

The CCRMS fee of 1–5% is a conservative allowance for protocol or treasury fees rather than a measured cost; Polkadot fees are not levied as a share of revenue, and the model charges CCRMS per-transaction costs separately.

G.2 Eight-Parameter Sensitivity Grid

Table G.3 varies each input parameter independently. The headline result is insensitive to pricing and conversion parameters and sensitive to the audience distribution.

Table G.3. Sensitivity of CCRMS Advantage to Individual Parameter Changes

Parameter varied	Range tested	Advantage over centralized
Audience μ	500 $\rightarrow$ 5,000	–0.2% to +36.3%
Audience σ	1.0 $\rightarrow$ 3.0	–26.3% to +39.1%
Content price floor	\$0.50 $\rightarrow$ \$5	+17.0% (unchanged)
Content price ceiling	\$20 $\rightarrow$ \$200	+17.0% (unchanged)
Subscription adoption α	1 $\rightarrow$ 5	+20.2% to +12.3%
PPV conversion α	0.5 $\rightarrow$ 3	+16.8% to +18.2%
Resale λ	0.05 $\rightarrow$ 0.5	+17.1% to +15.8%
Subscription period	1 $\rightarrow$ 24 months	+26.0% to +14.4%

G.3 CCRMS Fee Sweep

Table G.4 sweeps the CCRMS fee assumption. The mean-revenue advantage persists for any CCRMS fee below approximately 17%, more than three times the top of the assumed 1–5% range.

Table G.4. Sensitivity of CCRMS Advantage to Fee Assumption

CCRMS fee	Advantage over centralized	Win rate
1–5% (baseline)	+17.0%	28.1%
5%	+14.6%	26.9%
10%	+8.5%	23.6%
15%	+2.5%	19.8%
17%	+0.1%	18.1%
20%	–3.5%	15.4%

G.4 Discovery Boost Sweep

Table G.5 sweeps the centralized discovery bonus. The advantage reverses at approximately 1.9 times the assumed value, so discovery is the more influential assumption of the two.

Table G.5. Sensitivity of CCRMS Advantage to Centralized Discovery Assumption

Centralized discovery	Advantage over centralized	Win rate
0.25× (+125–750)	+36.3%	61.8%
0.5× (+250–1,500)	+29.2%	42.9%
1× (+500–3,000, baseline)	+17.0%	28.1%
1.5× (+750–4,500)	+6.9%	21.4%
2× (+1,000–6,000)	–1.6%	17.4%

G.5 Fine-Grained Crossover Buckets

Table G.6 shows the audience-size transition at fine resolution, establishing that the 50% crossover falls near 3,600 followers.

G.6 Reproduction Notes

To reproduce the fee sweep: set `PLATFORMS.ccrms.feeLow` and `feeHigh` to the same fixed value in `scripts/perf/monte-carlo-revenue.mjs` and re-run.

To reproduce the discovery sweep: multiply `PLATFORMS.centralized.discoveryAddLow` and `discoveryAddHigh` by the desired factor.

Both sweeps use `nCreators: 200`, `nIterations: 2_000` and `createRng(42)`, matching the existing `sensitivityAnalysis()` grid.

Table G.6. Fine-Grained CCRMS Win Rate Near the Crossover Audience Size

Base audience	Win rate vs centralized
1,500–2,000	16.3%
2,000–2,500	25.1%
2,500–3,000	34.3%
3,000–3,200	41.4%
3,200–3,400	45.1%
3,400–3,600	48.2%
3,600–3,800	51.9%
4,000–5,000	63.4%
7,500–10,000	94.2%

References

- 360iResearch. Blockchain for Content Monetization Market Size 2025-2030. Technical report, 360iResearch, 2025. URL <https://www.360iresearch.com/library/intelligence/blockchain-for-content-monetization>. Market Research Report.
- Amazon.com, Inc. ACX Royalty Rates. Audiobook Creation Exchange (ACX), 2025. URL <https://www.acx.com/royalties>. Accessed May 2026. Exclusive distribution plan pays 40% royalty to rights holders; Audible retains 60%.
- L. Ante. Smart contracts on the blockchain – A bibliometric analysis and review. SSRN Electronic Journal, 2020. URL https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3576393.
- Apple Inc. Apple Music Insights Royalty Rate. Apple for Artists Support, 2025. URL <https://artists.apple.com/support/1124-apple-music-insights-royalty-rate>. Accessed May 2026. Apple Music pays approximately 52% to labels; platform retains approximately 48%.
- M. Atzori. Blockchain Technology and Decentralized Governance: Is the State Still Necessary? *Journal of Governance and Regulation*, 6(1):45–62, 2017. doi: 10.22495/jgr_v6_i1_p5.
- P. Banerjee et al. Reliable, Fair and Decentralized Marketplace for Content Sharing Using Blockchain. arXiv:2009.11033, 2020. URL <https://arxiv.org/pdf/2009.11033>.
- Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. Addison-Wesley, 4th edition, 2021.
- A. Bigiotti, L. Mostarda, A. Navarra, A. Pinna, D. Sestili, R. Tonelli, and M. Vaccargiu. Blockchain Agnostic Protocols: An Analysis of the State of the Art. *ACM Computing Surveys*, 58(2):1–36, 2025. doi: 10.1145/3758089.
- K. Busch. Web3: A Proposed Blockchain-Based, Decentralized Web. Technical Report IF12075, Congressional Research Service, 2022. URL https://www.congress.gov/crs_external_products/IF/PDF/IF12075/IF12075.2.pdf.

- I. Buzu. Blockchain, smart contracts and copyright management disruption. SSRN, 2021. URL https://papers.ssrn.com/sol3/Delivery.cfm/SSRN_ID3759260_code3591422.pdf.
- M. A. Cabot-Nadal, M. Payeras-Capellà, M. Mut-Puigserver, and A. Soto-Fernández. Improving the Token ERC-721 Implementation for Selective Receipt: Rejectable NFTs. In *2022 6th International Conference on System Reliability and Safety (ICSRS)*, pages 243–250. IEEE, 2022. doi: 10.1109/ICSRS56243.2022.10067494.
- Y. Cao, J. Cao, D. Bai, L. Wen, Y. Liu, and R. Li. MAP the Blockchain World: A Trustless and Scalable Blockchain Interoperability Protocol for Cross-chain Applications. In *Proceedings of the ACM Web Conference 2025 (WWW '25)*. ACM, 2025. doi: 10.1145/3696410.3714867.
- Castweet Team. Castweet – A Multi-broadcast Platform Based on Blockchain Incentives for Everyone. Technical report, Castweet, 2019. URL <https://resources.cryptocompare.com/asset-management/17402/1728919640132.pdf>.
Whitepaper.
- M. Chegenizadeh, S. Rafati Niya, and C. J. Tessone. Multi-agent Privacy-Preserving Micro-payment Verification for Offline Resource-Constrained Devices Using Smart Contracts. In *Mathematical Research for Blockchain Economy (MARBLE 2024)*, Lecture Notes in Operations Research. Springer, 2024. doi: 10.1007/978-3-031-68974-1_7.
- K. Chen et al. A Trusted Reputation Management Scheme for Cross-Chain Transactions. *Sensors*, 23:6033, 2023.
- M. Cherdakov, A. Kudashkin, Y. Madhwal, and Y. Yanovich. Web3 Based Digital Rights Management in the Music Industry. In *Proceedings of the 2023 6th International Conference on Blockchain Technology and Applications (ICBTA)*. ACM, 2023. doi: 10.1145/3651655.3651667.
- R. F. Ciriello, A. C. G. Torbensen, M. R. P. Hansen, and C. Müller-Bloch. Blockchain-based digital rights management systems: Design principles for the music industry. *Electronic Markets*, 33(1), 2023. doi: 10.1007/s12525-023-00628-5.
- Coinbase and x402 Foundation. x402 Protocol: HTTP-native blockchain micropayments. CoinPaprika, 2025. URL <https://coinpaprika.com/education/x402-ecosystem-instant-blockchain-payments-for-apis-and-ai-agents/>.
- CommuniPass. Creator Monetization in 2026: The 5 Models That Actually Generate Recurring Revenue. CommuniPass Blog, 2026. URL <https://communipass.com/blog/creator-monetization-in-2026-the-5-models-that-actually-generate-recurring-revenues/>.
- P. Darshan, J. S. Rohan, R. Rajesh, M. Ruchitha, S. Kamath, and M. N. Manas. Intellectual Property Rights and Entrepreneurship in the NFT Ecosystem: Legal Frameworks, Business Models, and Innovation Opportunities. arXiv:2507.00172, 2025. URL <https://arxiv.org/html/2507.00172v1>.
- C. Davik. The Art Of NFTs: Copyright, Contracts, And The Fallacy Of Ownership. *Georgia Law Review*, 2025.
- Z. Deng, C. Tang, T. Li, P. Ablä, Q. Chen, W. Liang, and D. He. Enhancing Blockchain Cross-Chain Interoperability: A Comprehensive Survey. arXiv:2505.04934, 2025. URL <https://arxiv.org/abs/2505.04934>.

- I. Dondjio and A. Kazamias. A Blockchain Framework for Digital Asset Ownership and Transfer in Succession. In *20th European, Mediterranean, and Middle Eastern Conference on Information Systems (EMCIS 2023)*, volume 501 of *Lecture Notes in Business Information Processing*. Springer, 2024. doi: 10.1007/978-3-031-56478-9_7.
- A. Dreyer et al. Can I mint an NFT with that?: Avoiding right of publicity and trademark litigation risks in the brave new world of NFTs. Skadden Publications, 2021. URL <https://www.skadden.com/-/media/files/publications/2021/05/canimintannftwiththatavoidingrightofpublicityandtr.pdf>.
- Eurojust. Non-Fungible Tokens and Intellectual Property Rights – Can the use of NFTs lead to IP infringements? Technical report, European Union Agency for Criminal Justice Cooperation, 2024. URL <https://www.eurojust.europa.eu/sites/default/files/assets/eurojust-nfts-intellectual-property-rights-flyer.pdf>.
- European Securities and Markets Authority. MiCA Regulation: Markets in Crypto-Assets. Technical report, ESMA, 2025. URL <https://www.esma.europa.eu/esmas-activities/digital-finance-and-innovation/markets-crypto-assets-regulation-mica>. Full enforcement by July 2026.
- A. Garba et al. A digital rights management system based on a scalable blockchain. Copenhagen Business School Research, 2021. URL https://research-api.cbs.dk/ws/portalfiles/portal/93314574/abba_garba_et_al_a_digital_rights_management_system_based_on_a_scalable_blockchain_publishersversion.pdf.
- J. Gatto et al. NFT License Breakdown: Exploring Different Marketplaces and Associated License Issues. Law of the Ledger, 2021. URL <https://www.lawofthelledger.com/wp-content/uploads/sites/15/2021/09/NFT-License-Breakdown-Article-0921.pdf>.
- D. Gogel et al. DeFi Beyond the Hype: The Emerging World of Decentralized Finance. Technical report, Wharton Blockchain and Digital Asset Project, 2021. URL <https://wifpr.wharton.upenn.edu/wp-content/uploads/2021/05/DeFi-Beyond-the-Hype.pdf>.
- Google LLC. YouTube Partner Program: How ad revenue is shared. YouTube Help Center, 2025. URL <https://support.google.com/youtube/answer/72902>. Accessed May 2026. YouTube retains 45% of ad revenue; creators receive 55%.
- P. Goyal et al. Secure Incentivization for Decentralized Content Delivery. In *USENIX Workshop on Hot Topics in Edge Computing (HotEdge 2019)*, 2019. URL https://www.usenix.org/system/files/hotedge19-paper-goyal_0.pdf.
- Grand View Research. Digital Rights Management Market Size, Share & Trends Analysis Report. Technical report, Grand View Research, 2025. URL <https://www.grandviewresearch.com/industry-analysis/digital-rights-management-market>.
- A. Guadamuz. The EU’s Artificial Intelligence Act and copyright. *The Journal of World Intellectual Property*, 28:213–219, 2025. doi: 10.1111/jwip.12330.
- R. Habermeier et al. Polkadot’s messaging scheme. Web3 Foundation Research, Medium, 2021. URL <https://medium.com/web3foundation/polkadots-messaging-scheme-b1ec560908b7>.

- B. Hanneke, O. Hinz, J. Pfeiffer, and W. M. P. van der Aalst. The Internet of Value: Unleashing the Blockchain's Potential with Tokenization. *Business & Information Systems Engineering*, 66(4):411–419, 2024. doi: 10.1007/s12599-024-00883-6.
- A. R. Hevner, S. T. March, J. Park, and S. Ram. Design science in information systems research. *MIS Quarterly*, 28(1):75–105, 2004.
- ink! Alliance. Discontinuation of ink: Rust smart contract language. Polkadot Forum, 2026. URL <https://forum.polkadot.network/t/discontinuation-of-ink-rust-smart-contract-language/16849>. Posted January 27, 2026.
- K. Khan. Decentralized Video Streaming: Unleashing the Potential through Blockchain-Powered Platforms. *International Journal of Modern Research in Applied Physics (IJMRAP)*, 6(7), 2024.
- V. Kumar, I. Budhiraja, A. Jabbari, et al. Efficient blockchain interoperability design for cross-chain transactions in future internet-of-value. *Peer-to-Peer Networking and Applications*, 18:110, 2025. doi: 10.1007/s12083-025-01941-w.
- S. Lahiri et al. Multidisciplinary perspectives on Non-Fungible Tokens (NFTs): a comprehensive meta-analysis. *Digital Economy and Sustainable Development*, 4:3, 2026. doi: 10.1007/s44265-026-00078-9.
- W. Li, Z. Liu, J. Chen, Z. Liu, and Q. He. Towards blockchain interoperability: a comprehensive survey on cross-chain solutions. *Blockchain: Research and Applications*, 6(3):100286, 2025.
- S. Y. Lin et al. A survey of application research based on blockchain smart contract. *Wireless Networks*, 28: 635–690, 2022. doi: 10.1007/s11276-021-02874-2.
- S. L. Madapati and N. R. Pradhan. Decentralizing video copyright protection: a novel blockchain-enabled framework with performance evaluation. *Frontiers in Artificial Intelligence*, 8, 2025.
- Magic Eden. Magic Eden Marketplace Fees. Magic Eden Help Center, 2025. URL <https://help.magiceden.io>. Accessed May 2026. Magic Eden charges a 2% marketplace fee on NFT sales.
- H. Mao et al. A Survey on Cross-Chain Technology: Challenges, Development, and Prospect. 2022.
- D. Mareckova. *Blockchain and Collective Rights Management of Copyright and Related Rights at the Global Level: The Case of the Music Industry*. PhD thesis, OAPEN, 2024. URL https://library.oapen.org/bitstream/handle/20.500.12657/94575/oa_pdf-057-1731486746.pdf.
- F. Maric, B. Scholz, and P. Subotic. Formal Verification of a Fail-Safe Cross-Chain Bridge. In *6th International Workshop on Formal Methods for Blockchains (FMBC 2025)*, volume 129 of *OASICS*, 2025. URL <https://drops.dagstuhl.de/entities/document/10.4230/OASICS.FMBC.2025.8>.
- MarketsandMarkets. Digital Rights Management Market by Application. Technical report, MarketsandMarkets Research, July 2025. URL <https://www.marketsandmarkets.com/Market-Reports/digital-rights-management-market-152806525.html>.
- J. C. Mendoza-Tello, J. J. Sánchez-Lucas, and J. G. Orosco-Pantoja. A Model to Support the Music Streaming Service Using Blockchain-Based Smart Contracts. In *Applied Technologies (ICAT 2023)*, volume 2051 of *Communications in Computer and Information Science*. Springer, 2024. doi: 10.1007/978-3-031-58950-8_10.

- Mordor Intelligence. Digital Rights Management (DRM) Market – Size, Share & Industry Analysis. Technical report, Mordor Intelligence, January 2026. URL <https://www.mordorintelligence.com/industry-reports/digital-rights-management-drm-market>.
- D. Morháč, V. Valaštin, K. Košťál, and I. Kotuliak. ParaSpell XCM SDK: A New Protocol for Interoperability in Polkadot Paraverse. In *2023 Fifth International Conference on Blockchain Computing and Applications (BCCA)*, pages 569–576. IEEE, 2023a. doi: 10.1109/BCCA58897.2023.10338906.
- D. Morháč, V. Valaštin, K. Košťál, and I. Kotuliak. Enhancing XCMP Interoperability Across Polkadot Paraverse. In *2023 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, pages 1–3. IEEE, 2023b. doi: 10.1109/ICBC56567.2023.10174872.
- D. Morháč, K. Košťál, V. Valaštin, et al. Unispell: universal adapter for interoperability in Polkadot Paraverse. *Cluster Computing*, 28:311, 2025. doi: 10.1007/s10586-025-05183-6.
- M. Munson. A Blockchain Controlled Content Fabric. Video Services Forum, 2019. URL https://static.vsf.tv/Meetings/2019/2019_DC/Munson_BlockchainFabric.pdf.
- M. Munson et al. Blockchain Content Fabric for Multi-Chain Content Ownership and Distribution. Technical report, Eluvio, 2022. URL <https://eluv.io/3e39198441b1d578bc0f.pdf>.
- M. Murray. NFT Ownership And Copyrights. *Indiana Law Review*, 56:367, 2022.
- Music Business Research. The Music Streaming Economy Part 14: The Artists’ Share of the Music Streaming Pie. Music Business Research Blog, 2024. URL <https://musicbusinessresearch.wordpress.com/2024/09/16/the-music-streaming-economy-part-14-the-artists-share-of-the-music-streaming-pie/>. Accessed May 2026.
- OpenSea. OpenSea Fees. OpenSea, 2025. URL <https://opensea.io/fees>. Accessed May 2026. OpenSea charges a 2.5% protocol fee on secondary sales; creator royalties are set separately by the collection creator.
- P. Orero et al. The Visible Subtler: Blockchain Technology for Right Management and Minting. *Open Research Europe*, 3:26, 2023.
- O. Osemwegie. Decentralized Media Distribution: Exploring blockchain’s role in copyright protection, monetization, and content ownership rights. *International Journal of Mechanical Technology and Marine Engineering*, 2025.
- Parity Technologies. Polkadot Parachain Slots. Polkadot Blog, 2021. URL <https://polkadot.com/blog/polkadot-parachain-slots/>. Describes Polkadot’s support for up to 100 parachain slots and additive parallel throughput.
- Parity Technologies. ink! 3.0: Parity’s Rust-based language for WASM smart contracts gets a major update. Parity Blog, 2022a. URL <https://www.parity.io/blog/ink-3-0-paritys-rust-based-language-gets-a-major-update>.
- Parity Technologies. What is Parity’s ink!? Parity Blog, 2022b. URL <https://www.parity.io/blog/what-is-paritys-ink>.

- Parity Technologies. Polkadot Upgrade 2025: What You Need to Know. Parity Blog, 2025. URL <https://www.parity.io/blog/polkadot-upgrade-2025-what-you-need-to-know>.
- David L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972. doi: 10.1145/361598.361623.
- S. S. Patil et al. Blockchain Based Multimedia Content Protection Using Encryption in DRM (Digital Rights Management). *International Journal of Innovative Research in Technology (IJIRT)*, 2022.
- Polkadot Network. Polkadot Roundup 2025. Medium, 2025. URL <https://medium.com/polkadot-network/polkadot-roundup-2025-3c3c71c7e9c4>.
- L. Quan et al. Research and application of digital collection smart contract method based on Blockchain. 2024.
- I. S. Rao, M. L. M. Kiah, M. M. Hameed, and Z. A. Memon. Scalability of blockchain: a comprehensive review and future research direction. *Cluster Computing*, 27:5547–5570, 2024. doi: 10.1007/s10586-023-04257-7.
- E. Reggianini. Standardizing On-chain IP Rights Management. Technical report, BlockStand, 2025. URL https://blockstand.eu/blockstand/uploads/2025/05/Standardizing-On-chain-IP-Rights-Management_Reggianini.pdf.
- K. Ren, N.-M. Ho, D. Loghin, T.-T. Nguyen, B. C. Ooi, Q.-T. Ta, and F. Zhu. Interoperability in Blockchain: A Survey. *IEEE Transactions on Knowledge and Data Engineering*, 35(12):12750–12769, 2023. doi: 10.1109/TKDE.2023.3275220.
- A. Shah. Automating Royalties: A Framework of Smart Contracts for Cross-Platform Content Revenue. *International Journal for Multidisciplinary Research (IJFMR)*, 2025.
- R. Singh et al. Insights into research on blockchain for smart contracts: A bibliometric analysis. *Multimedia Tools and Applications*, 83:1–25, 2024. doi: 10.1007/s11042-023-15647-1.
- Snowfork. Snowbridge V2 Is Live! Polkadot Forum, 2025a. URL <https://forum.polkadot.network/t/snowbridge-v2-is-live/15844>. Posted November 7, 2025.
- Snowfork. Snowbridge 2025/2026 Stabilized. Polkasembly Proposal #3313, 2025b. URL <https://polkadot.polkasembly.io/post/3313>.
- Spotify AB. Understanding Spotify Royalties. Spotify for Artists Support, 2025. URL <https://support.spotify.com/us/artists/article/understanding-spotify-royalties/>. Accessed May 2026. Spotify pays approximately 70% of revenue to rights holders.
- Steem Team. Steem, an incentivized, blockchain-based, public content platform. Technical report, Steem, 2025. URL <https://steem.com/SteemWhitePaper.pdf>. Whitepaper.
- Story Foundation. Story Mainnet Launch. CoinDesk, 2025. URL <https://www.coindesk.com/tech/2025/02/13/story-protocol-launches-to-let-people-to-register-ip-and-get-paid-for-it/>. Posted February 13, 2025.

- U.S. Department of Justice and Federal Trade Commission. Horizontal Merger Guidelines. Technical report, U.S. Department of Justice, 2010. URL <https://www.justice.gov/atr/horizontal-merger-guidelines-08192010>. Accessed September 2026.
- USPTO. Non-Fungible Tokens and Intellectual Property: A report to Congress. Technical report, U.S. Patent and Trademark Office, 2024. URL <https://www.copyright.gov/policy/nft-study/Joint-USPTO-USCO-Report-on-NFTs-and-Intellectual-Property.pdf>.
- M. Wamugo. Content monetization: non-fungible tokens as a new revenue stream for the media sector in Kenya. Master's thesis, Aga Khan University, 2024. URL https://ecommons.aku.edu/cgi/viewcontent.cgi?article=3232&context=theses_dissertations.
- Z. Wang, W. Feng, M. Huang, S. Feng, S. Mo, and Y. Li. Blockchain-based information security protection mechanism for the traceability of intellectual property transactions. *Sensors*, 25(10):3064, 2025.
- G. Wood. Polkadot: Vision for a heterogeneous multi-chain framework. Technical report, Web3 Foundation, 2016. URL <https://polkadot.network/whitepaper>. White Paper.
- G. Wood. XCM: The cross-consensus message format. Polkadot Blog, 2022. URL <https://www.polkadot.network/blog/xcm-the-cross-consensus-message-format>.
- G. Wood. JAM: The Join-Accumulate Machine (Gray Paper). Technical report, Web3 Foundation, 2024. URL <https://wiki.polkadot.com/learn/learn-jam-chain/>. Version 0.7.1+, updated through 2025.
- R. Xie and Y. Liu. Research on copyright protection of digital works based on multi-blockchain. *Recent Advances in Computer Science and Communications*, 15(8):1126–1134, 2022. doi: 10.2174/2666255815666220204092947.
- R. Xie and M. Tang. A digital resource copyright protection scheme based on blockchain cross-chain technology. *Heliyon*, 10(16):e34781, 2024. doi: 10.1016/j.heliyon.2024.e34781.
- X. Yi, Y. Zhou, Y. Lin, et al. Digital rights management scheme based on redactable blockchain and perceptual hash. *Peer-to-Peer Networking and Applications*, 16:2630–2648, 2023. doi: 10.1007/s12083-023-01552-3.
- Z. Yin et al. Bool Network: An Open, Distributed, Secure Cross-chain Notary Platform. IACR ePrint 2022/1290, 2022. URL <https://eprint.iacr.org/2022/1290.pdf>.